AZURE CAPSTONE PROJECT

Intelligent Serverless FinOps & Automated Compliance Remediation Orchestrator

Project documentation: design, step-by-step implementation, testing and evidence

Student	Mohammed Shamnan
Project type	Azure Capstone Project
Proposal date	31 August 2026
Documentation date	October 2026
Submitted to (mentor)	Nidha
Resource group	rg-finops-compliance-prod-Shamnan

Contents

1. Project Overview
- 1.1 Objective
- 1.2 Problem addressed
- 1.3 What the system does
- 1.4 Design principles

2. Architecture and Data Flow
- 2.1 End-to-end flow
- 2.2 Azure services used
- 2.3 Decision logic

3. Resource Inventory

4. Prerequisites and Conventions
- 4.1 What you need
- 4.2 Conventions used in this guide

5. Implementation Guide
- Phase 1: Resource group and environment
- Phase 2: Azure Cosmos DB audit store
- Phase 3: Azure Key Vault
- Phase 4: Azure Policy guardrail
- Phase 5: Function App (remediation engine)
- Phase 6: Event Grid system topic and subscription
- Phase 7: Identities and permissions (RBAC)
- Phase 8: Logic App approval workflow
- Phase 9: Store the Logic App URL in Key Vault
- Phase 10: Audit logging to Cosmos DB
- Phase 11: Function logic in Code + Test
- Phase 12: Automatic delete option
- Phase 13: Monitoring and alerts (Azure Monitor)
- Phase 14: Microsoft Defender for Cloud
- Phase 15: Azure Container Instances vulnerability scan
- Phase 16: Admin dashboard (Azure App Service)
- Phase 17: Resource locks

6. Testing and Validation
- 6.1 Evidence for the main scenarios

7. Troubleshooting Log

8. Security Design

9. Cost and Clean-up
- 9.1 Cost behaviour
- 9.2 Clean-up after the project

10. Limitations and Future Work
- 10.1 Current limitations
- 10.2 Future work

11. Conclusion

Appendix A: Final run.ps1 (EventGridTrigger1)

Appendix B: Sample Test Events

Appendix C: Dashboard Reference Code (app.py)

Appendix D: Screenshot Evidence Checklist

1. Project Overview

1.1 Objective

This project builds an **automated governance engine** for Microsoft Azure. It continuously watches a resource group, detects configuration drift as soon as a resource is created or changed, and responds without manual effort: low-risk problems are corrected automatically, high-risk resources are sent to a human for approval, and unwanted public IP addresses can be removed automatically under strict safety rules. Every decision is recorded in an audit ledger.

1.2 Problem addressed

- Resources are created without mandatory tags, so costs cannot be allocated to a department.
- Risky resources (public IP addresses, virtual machines) appear without review and keep costing money.
- Manual checking is slow, inconsistent and leaves no reliable history.

1.3 What the system does

- **Detects** resource creation and change events in real time using Azure Event Grid.
- **Auto-remediates** missing `Department` tags by applying `Department = IT`.
- **Requests human approval** by email (Approve / Reject) for high-risk resources such as virtual machines.
- **Deletes unapproved public IPs automatically** when the optional automatic-delete mode is enabled, with a dry-run mode, a `Keep` tag opt-out and a kill switch.
- **Records every action** in an Azure Cosmos DB audit ledger and shows it on an admin dashboard.
- **Raises alerts** by email when a remediation run fails, and keeps secrets in Azure Key Vault.
- **Runs periodic security scans** with a container-based vulnerability scanner.

1.4 Design principles

Principle	How it is applied
Serverless and event-driven	No servers to manage. Functions, Logic Apps and Cosmos DB serverless cost almost nothing when idle.
Least privilege	The function identity holds only Tag Contributor and a custom role that can delete public IPs and nothing else.
Human in the loop	High-risk resources need an explicit Approve or Reject from an administrator.
Safe by default	Automatic deletion is off until enabled, has a dry-run mode, ignores resources tagged Keep=true and is limited to one resource type.
Auditability	Every detection, approval request, skip and deletion is stored as a record in Cosmos DB.

2. Architecture and Data Flow

2.1 End-to-end flow

1. A user or process **creates or changes a resource** in the project resource group.
2. Azure **Event Grid** publishes a `ResourceWriteSuccess` event for that resource.
3. The **Azure Function** (`EventGridTrigger1`) receives the event and decides what to do, using its managed identity.
4. **Low risk** (for example a missing tag): the function applies the `Department = IT` tag itself.
5. **High risk** (for example a virtual machine): the function calls a **Logic App**, which emails an administrator with Approve and Reject buttons and waits for the answer.
6. **Public IP** with automatic-delete mode on: the function deletes it unless it carries the `Keep = true` tag.
7. Each outcome is written to the **Cosmos DB AuditLogs** container, shown on the **dashboard**, and failures trigger an **Azure Monitor** alert.

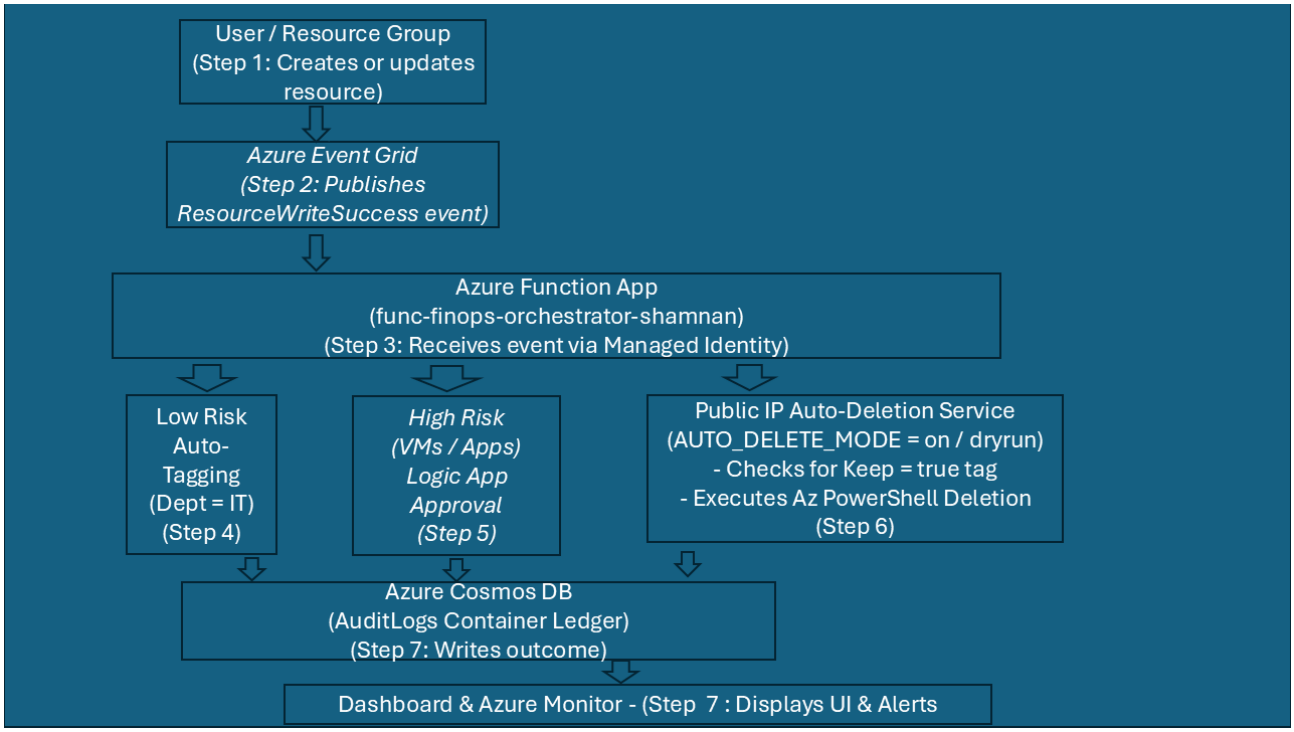


Figure 1: Architecture diagram of the solution

2.2 Azure services used

#	Service	Role in the project
1	Azure Event Grid	Captures resource-group events in real time and triggers the function.
2	Azure Functions	Runs the compliance logic: tagging, approval requests, automatic delete, audit logging.
3	Azure Policy	Defines the organisational standard (required Department tag) and compliance reporting.

#	Service	Role in the project
4	Microsoft Defender for Cloud	Assesses security posture and provides a secure score and recommendations.
5	Azure Cosmos DB	Stores the audit ledger of violations and actions (serverless).
6	Azure Logic Apps	Runs the human-in-the-loop approval workflow by email.
7	Azure Key Vault	Stores the Logic App trigger URL as a secret, read by the function through a Key Vault reference.
8	Azure Monitor	Application Insights logs and an alert rule for failed runs.
9	Azure App Service	Hosts the administrative dashboard that reads the audit ledger.
10	Azure Container Instances	Runs a batch vulnerability scanner on demand.

2.3 Decision logic

The function evaluates each event in the order below and stops at the first rule that applies.

Order	Condition	Behaviour
1	Event is not ResourceWriteSuccess	Ignored.
2	Resource is outside the project resource group	Ignored (scope guard).
3	Event comes from a deployment operation	Ignored.
4	Public IP and AUTO_DELETE_MODE is dryrun or on	Keep=true: skip and log. dryrun: log 'would delete', nothing deleted. on: delete and log.
5	Public IP or virtual machine (all other cases)	Send approval request to the Logic App and log it.
6	Any other resource	If the Department tag is missing, add Department=IT and log it.

3. Resource Inventory

All resources live in one resource group so that they can be managed, monitored and cleaned up together.

Resource name	Type	Region / notes
rg-finops-compliance-prod-Shamnan	Resource group	South India. Tag Department = IT.
func-finops-orchestrator-shamnan	Function App	PowerShell 7.6, Consumption, Windows. Central US.
func-finops-orchestrator-shamnan (Application Insights)	Application Insights	Linked to the Function App.
stfinops99shamnan	Storage account	Function App host storage. Central US.
cosmos-finops-audit-shamnan	Azure Cosmos DB for NoSQL	Serverless. West US 2. Database ComplianceDB, container AuditLogs, partition key /resourceId.
vault-finops-shamnan	Key Vault	Azure RBAC permission model. West US 2.
finops-system-topic	Event Grid system topic	Source: the resource group. Global.
sub-finops-remediation	Event subscription	Delivers events to EventGridTrigger1.
logic-finops-approval-shamnan	Logic App (Consumption)	South India.
plan-finops-dashboard	App Service plan	Free tier. South India.
app-finops-dashboard-shamnan	Web App (Python)	Admin dashboard. Enter your web app name.
Require-Department-Tag	Policy assignment	Subscription scope. Tag Department, value IT.
Public IP Deleter (FinOps)	Custom RBAC role	Read and delete public IP addresses only.
shamnandemo	Storage account	Test resource used to demonstrate auto-tagging. South India.
alert-finops-failed-remediation, ag-finops-alerts, aci-trivy-scan	Monitor alert, action group, ACI	Enter the names you used. Container group is temporary and deleted after the scan.

4. Prerequisites and Conventions

4.1 What you need

- An active Azure subscription (free credits or student credits are enough).
- Access to the Azure portal and Azure Cloud Shell (choose **PowerShell**, and **No storage account required** when asked).
- A mailbox that can send Outlook.com approval emails, and an email address to receive them.
- Owner or equivalent rights on the resource group, so that roles and locks can be managed.

4.2 Conventions used in this guide

- Replace `<subscription-id>` with your own subscription ID wherever it appears.
- Every resource is tagged `Department = IT` at creation (the function adds the tag automatically if it is missing).
- Never publish the Logic App trigger URL: it contains a secret key. Store it only in Key Vault.
- Screenshot boxes in this document are placeholders. Paste your own screenshot into each dashed box and keep the caption.

5. Implementation Guide

The phases below follow the order in which the system was built. Each phase states its purpose, the exact steps, and the screenshot that serves as evidence.

Phase 1: Resource group and environment

Purpose: *Create one container for every resource of the project.*

1. Azure portal > **Resource groups** > **Create**.
2. Subscription: *Azure subscription 1*. Resource group name: `rg-finops-compliance-prod-Shamnan`. Region: **South India**.
3. Tags tab: add `Department` = `IT`.
4. Click **Review + create**, then **Create**.

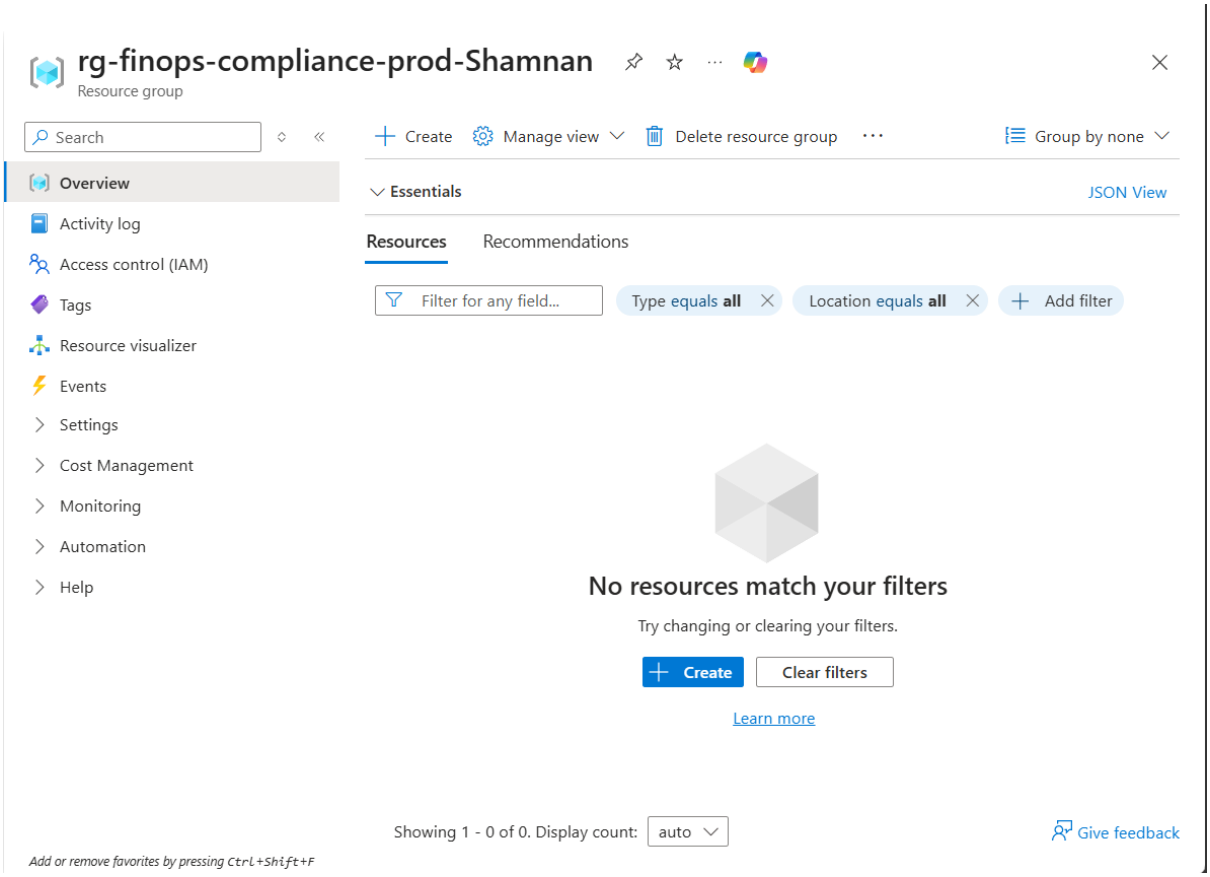


Figure 2: Empty resource group before any resource was deployed

Phase 2: Azure Cosmos DB audit store

Purpose: *Create the database that holds the audit ledger.*

1. **Create a resource** > **Azure Cosmos DB** > **Azure Cosmos DB for NoSQL** > **Create**.
2. Resource group: the project group. Account name: `cosmos-finops-audit-shamnan`. Location: West US 2 (chosen for availability on the subscription).
3. Capacity mode: **Serverless** (pay per request, no idle cost).
4. Tags: `Department` = `IT`. Click **Review + create**, then **Create**, then **Go to resource**.

5. Open **Data Explorer** > **New Container**. Database id: **Create new** `ComplianceDB`. Container id: `AuditLogs`. Partition key: `/resourceId`. Click **OK**.

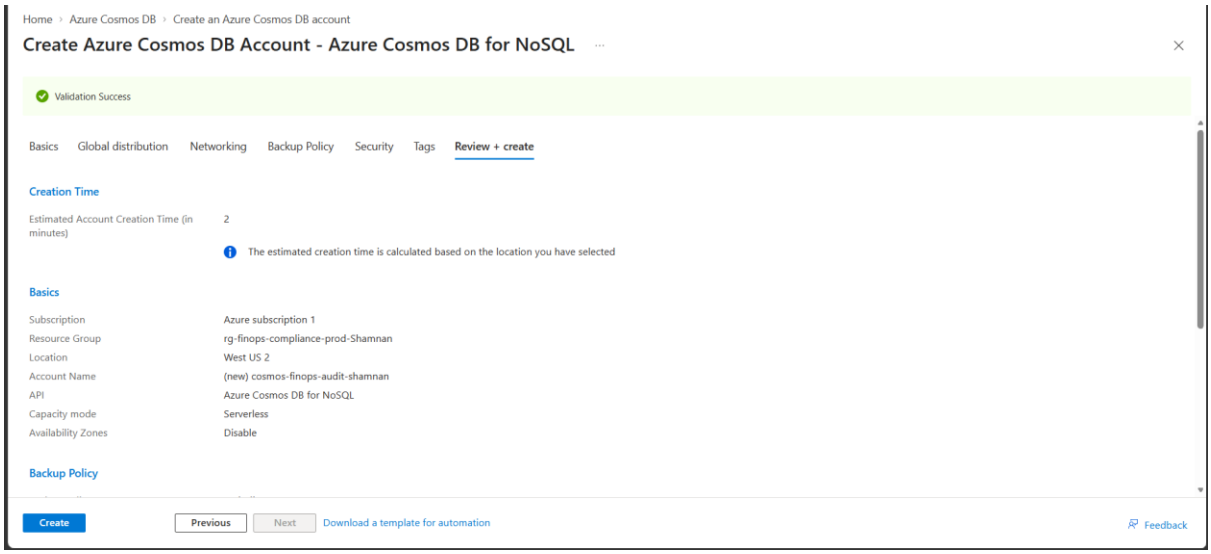


Figure 3: Cosmos DB review page showing Serverless capacity mode and account name

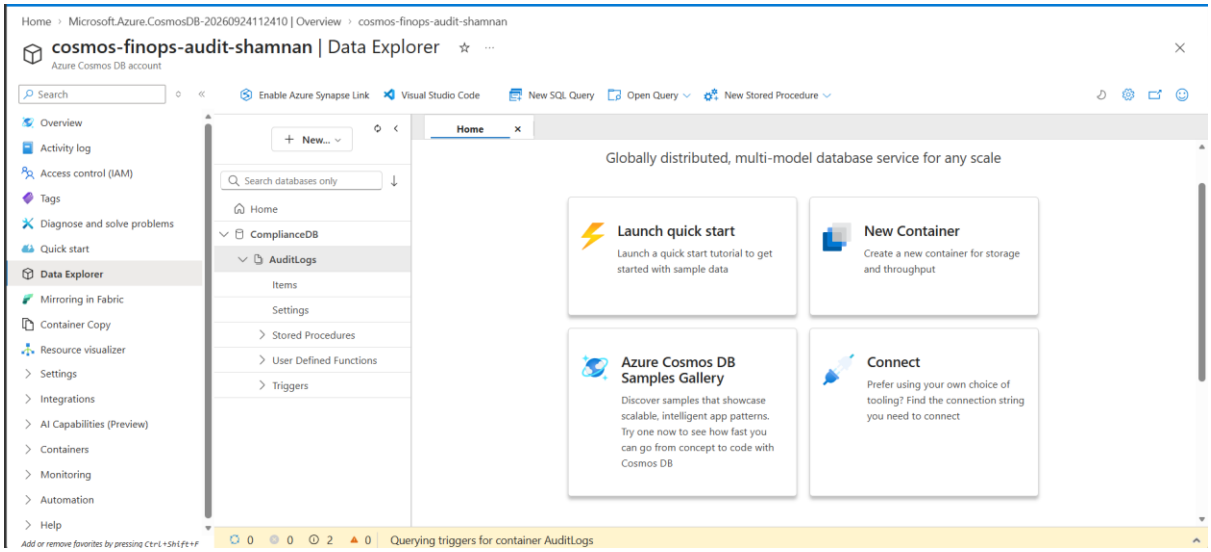


Figure 4: Data Explorer showing ComplianceDB > AuditLogs

Each audit record has the following fields:

Field	Meaning
id	Unique identifier (GUID) generated for every record.
resourceId	Full Azure resource ID. This is the partition key.
resourceGroup	Resource group extracted from the resource ID.
violation	What was detected, for example MissingDepartmentTag or Unapproved public IP.
action	What the system did: AutoRemediated, ApprovalRequested, WouldDelete, DeleteSkipped, AutoDeleted.
status	Outcome: Success, Pending, Failed, DryRun, KeepTag.
timestamp	UTC time of the record (ISO 8601).

Phase 3: Azure Key Vault

Purpose: *Create a secure place for secrets so that no secret is stored in code or plain settings.*

1. **Create a resource** > **Key Vault** > **Create**.
2. Resource group: the project group. Name: `vault-finops-shamnan`. Region: West US 2. Pricing tier: **Standard**.
3. Access configuration: **Azure role-based access control** (recommended). Soft-delete stays enabled.
4. Tags: `Department` = `IT`. Click **Review + create**, then **Create**.

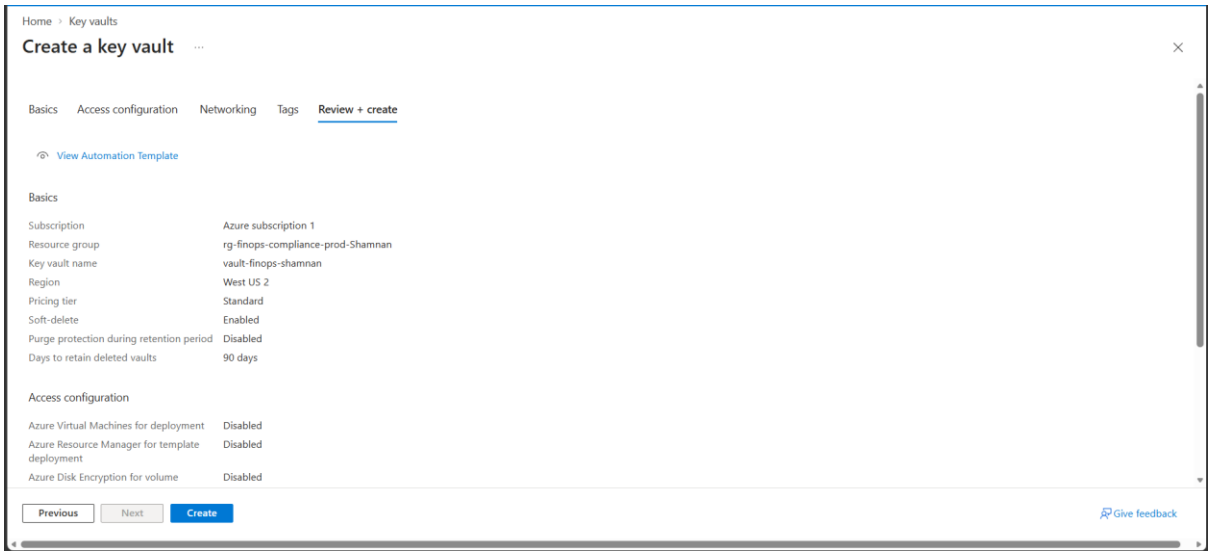


Figure 5: Key Vault review page (name, region, Standard tier)

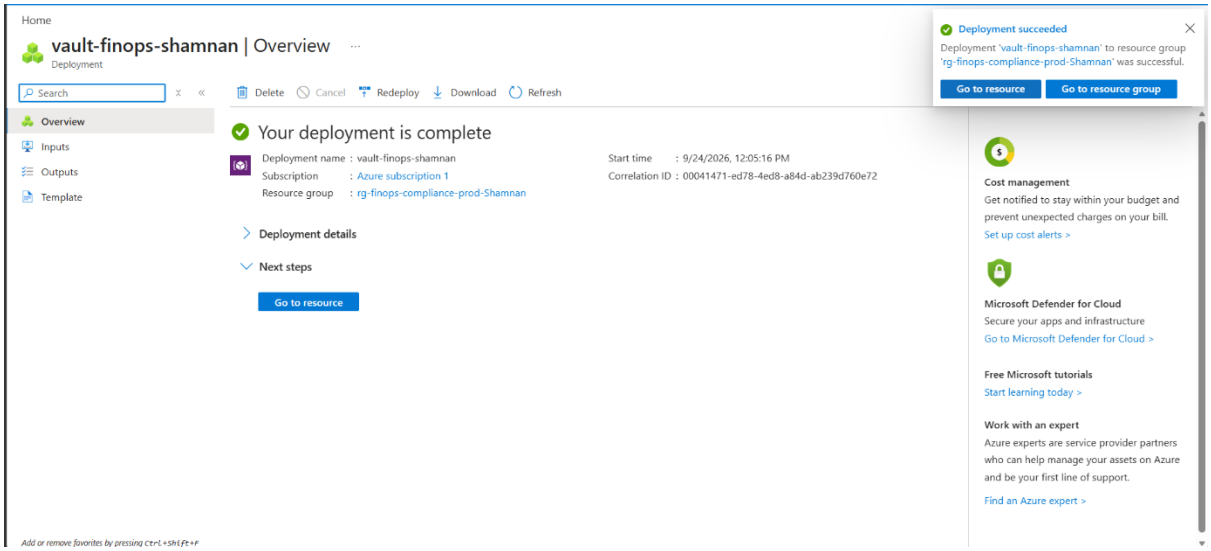


Figure 6: Key Vault deployment succeeded

> The vault stays empty until Phase 9, where the Logic App URL is stored as a secret.

Phase 4: Azure Policy guardrail

Purpose: *Define the organisational standard that every resource must carry a Department tag.*

1. Portal search > **Policy** > **Assignments** > **Assign policy**.

2. Scope: *Azure subscription 1*. Policy definition: **Require a tag and its value on resources**. Assignment name: `Require-Department-Tag`.
3. Parameters: Tag Name = `Department`, Tag Value = `IT`.
4. Click **Review + create**, then **Create**.
5. Later, open **Policy > Compliance** to see which resources are compliant.

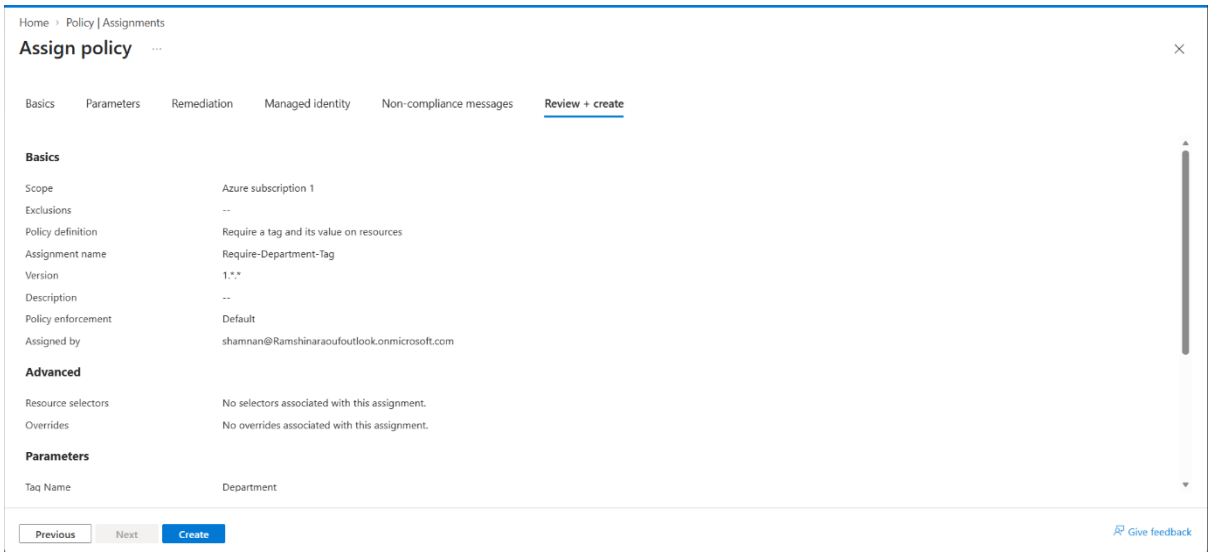


Figure 7: Assign policy review page (Require-Department-Tag)

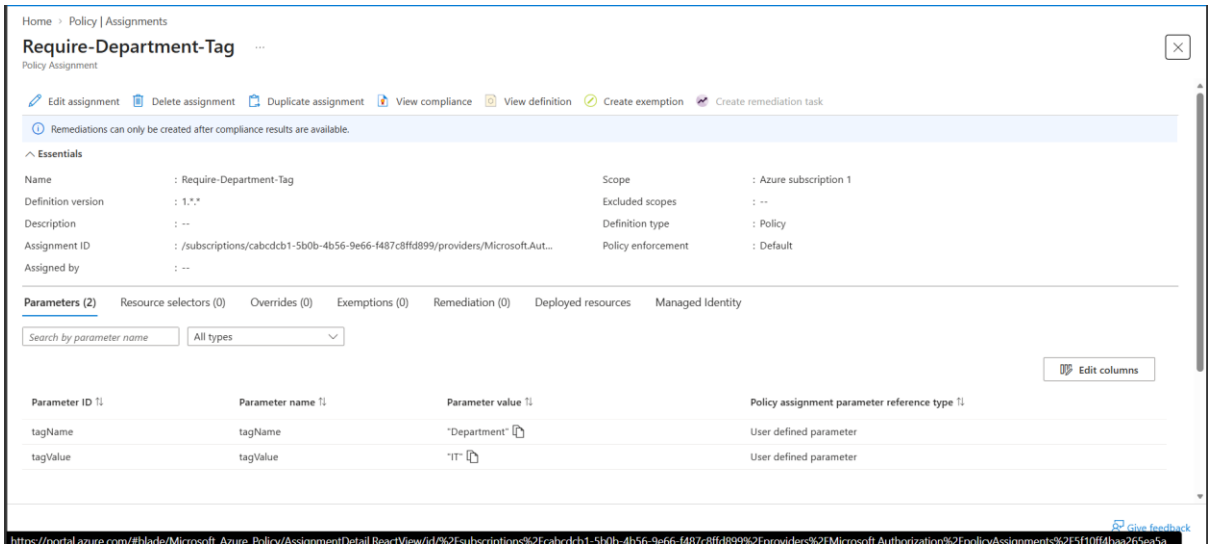


Figure 8: Policy assignment details with parameters tagName and tagValue

> **Note:** Azure Policy defines and reports the standard. The Azure Function performs the automatic correction. State in your presentation which effect your policy uses (open **View definition** and read the `effect` line) and how it fits with the function.

Phase 5: Function App (remediation engine)

Purpose: *Create the serverless engine that receives events and acts on them.*

1. **Create a resource** > **Function App** > **Consumption**.
2. Resource group: the project group. Name: `func-finops-orchestrator-shamnan`. Runtime stack: **PowerShell Core**, version **7.6**. Operating system: **Windows**. Region: Central US.

3. Monitoring tab: **Enable Application Insights = Yes**. Without it no invocation logs are available.
4. Tags: `Department` = `IT`. Click **Review + create**, then **Create**.
5. Open the Function App > **Settings > Identity > System assigned** > **On** > **Save** > **Yes**. This is the managed identity that the function uses to call Azure.
6. **Functions > Create > Azure Event Grid trigger**. Name it `EventGridTrigger1`.
7. **Functions > App files**: open `requirements.psd1` and replace its content with the block below. Save.

```
@{
    'Az.Accounts'  = '3.*'
    'Az.Resources' = '7.*'
}
```

1. In `host.json` confirm that managed dependency is enabled, then Save:

```
"managedDependency": {
    "Enabled": true
}
```

1. In `profile.ps1` confirm that the managed identity login is present (it is there by default):

```
if ($env:MSI_SECRET) {
    Disable-AzContextAutosave -Scope Process | Out-Null
    Connect-AzAccount -Identity
}
```

1. From **Overview**, click **Restart**. The first run afterwards downloads the PowerShell modules, which can take several minutes.

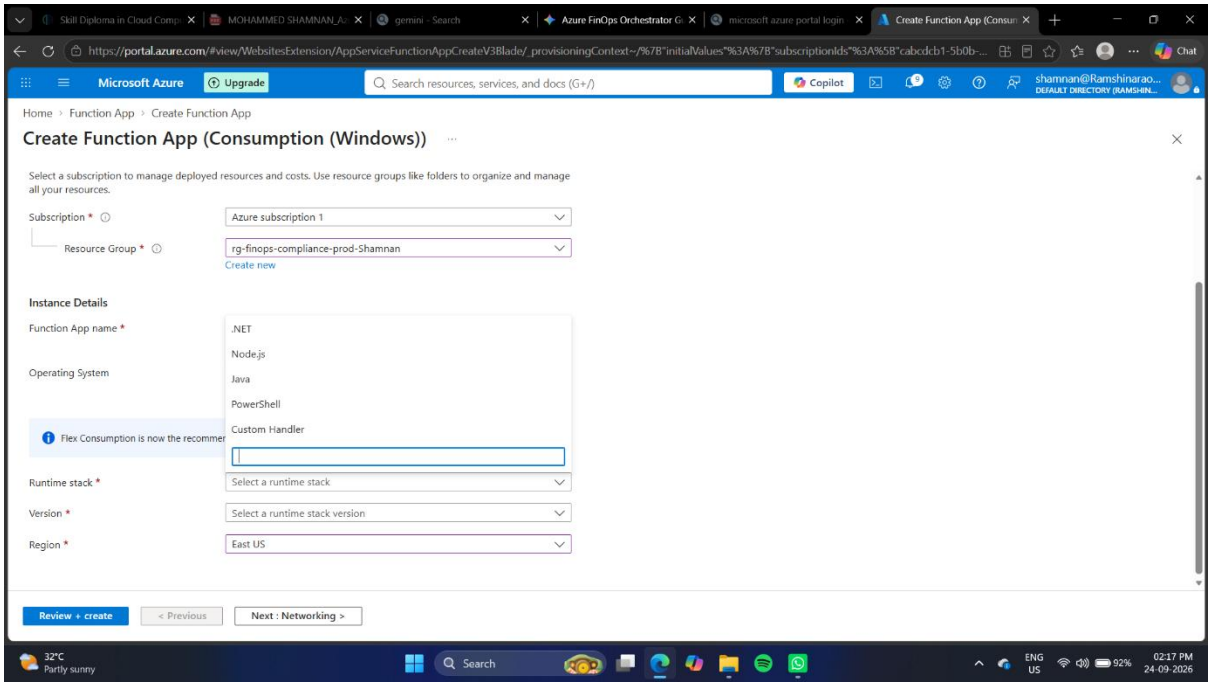


Figure 9: Function App review page (PowerShell 7.6, Consumption, Windows)

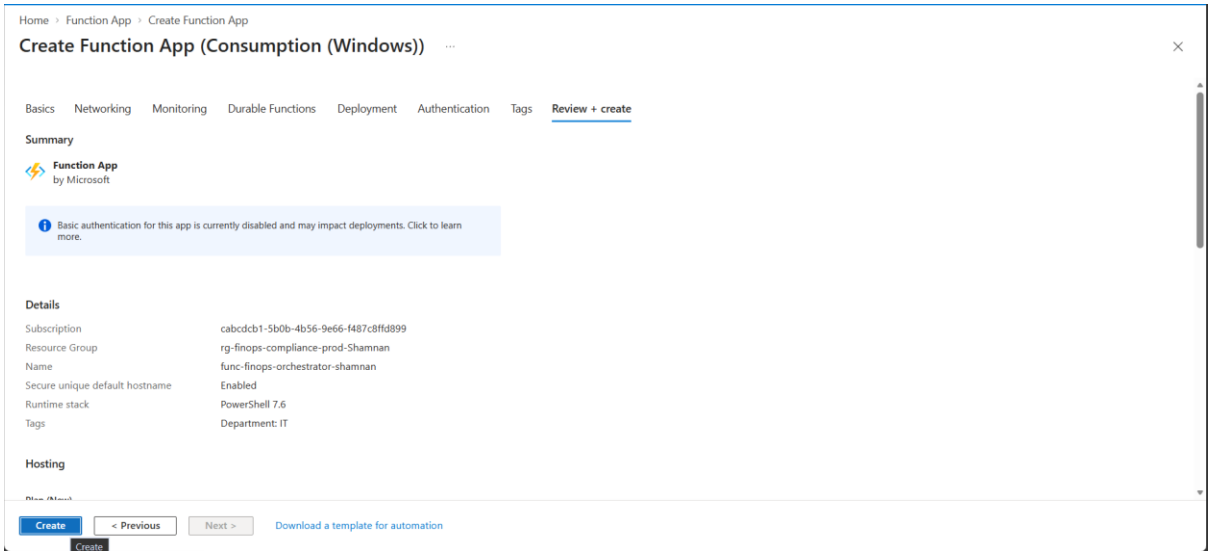


Figure 10: Function App overview with Department: IT tag

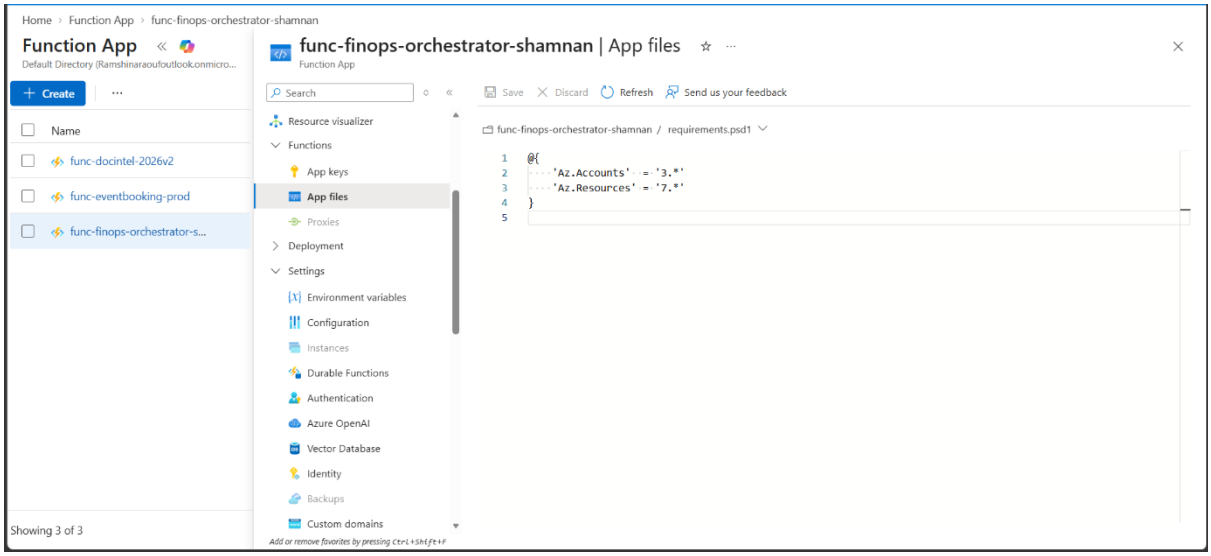


Figure 11: requirements.psd1 with Az.Accounts and Az.Resources enabled

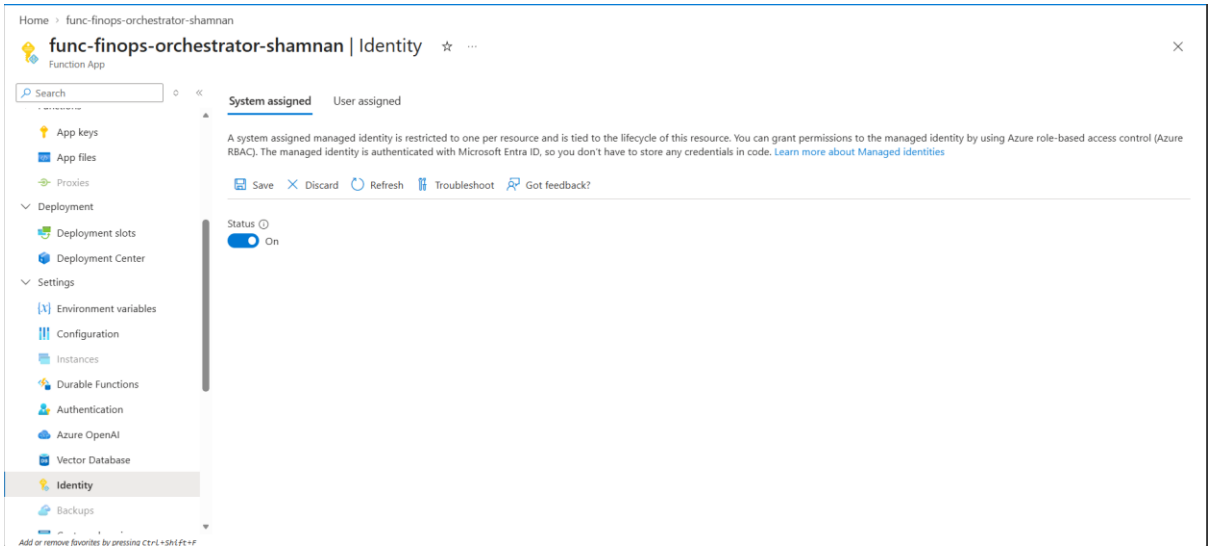


Figure 12: Identity page: system assigned managed identity switched On

> **Why only two modules?** The full Az bundle is very large and often fails to load in a Consumption function. Az.Accounts and Az.Resources contain every command this project needs (Get-AzTag, Update-AzTag, Get-AzAccessToken).

Phase 6: Event Grid system topic and subscription

Purpose: *Send every resource event in the project resource group to the function.*

1. **Create a resource** > **Event Grid System Topic**. Topic type: **Resource Groups**. Source: the project resource group. Name: `finops-system-topic`. Tags: `Department` = `IT`. Create.
2. Open the topic > **+ Event Subscription**. Name: `sub-finops-remediation`. Event schema: **Event Grid Schema**.
3. Filter to event types: keep **Resource Write Success** (the function ignores all other event types).
4. Endpoint type: **Azure Function**. Click **Configure an endpoint**, select `func-finops-orchestrator-shamnan` and the function `EventGridTrigger1`, then **Confirm Selection**.
5. Click **Create**. The subscription shows **Succeeded** on the topic overview page.

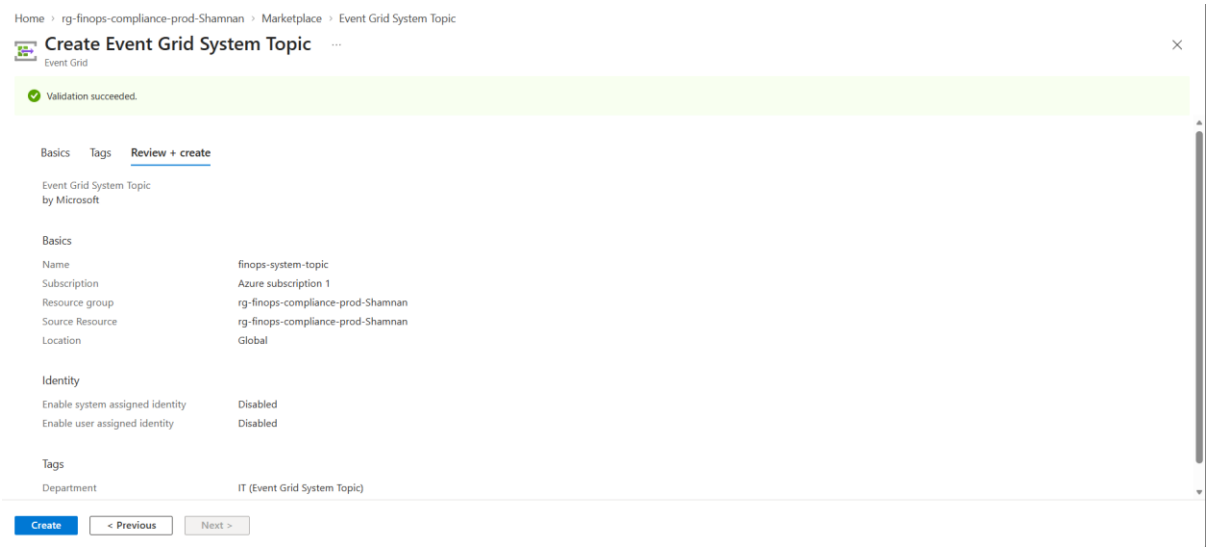


Figure 13: Event Grid system topic review page (source = resource group)

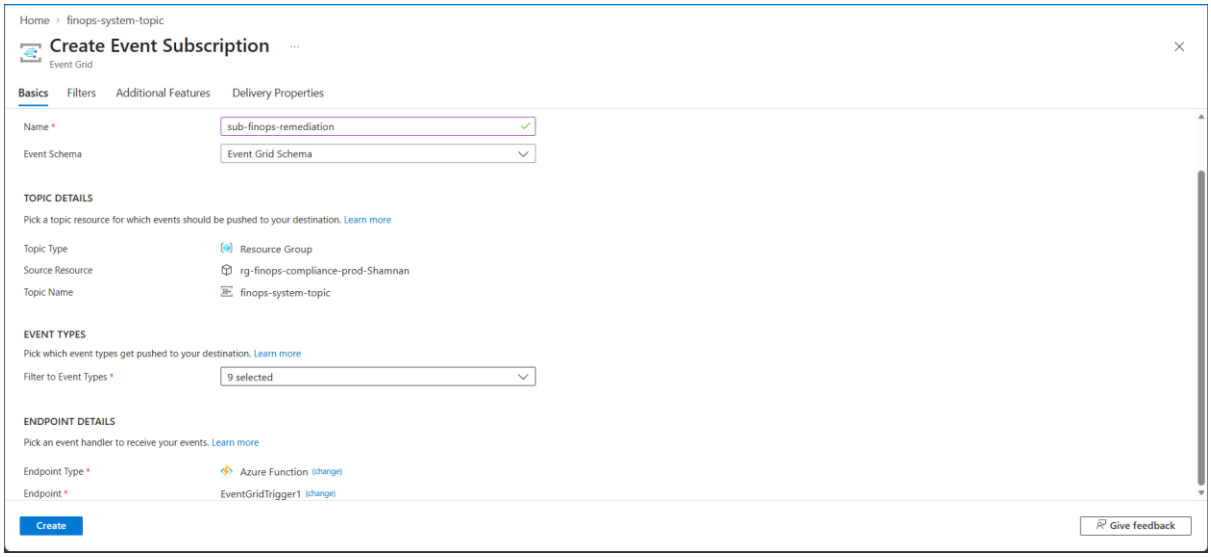


Figure 14: Create Event Subscription form with Azure Function endpoint EventGridTrigger1

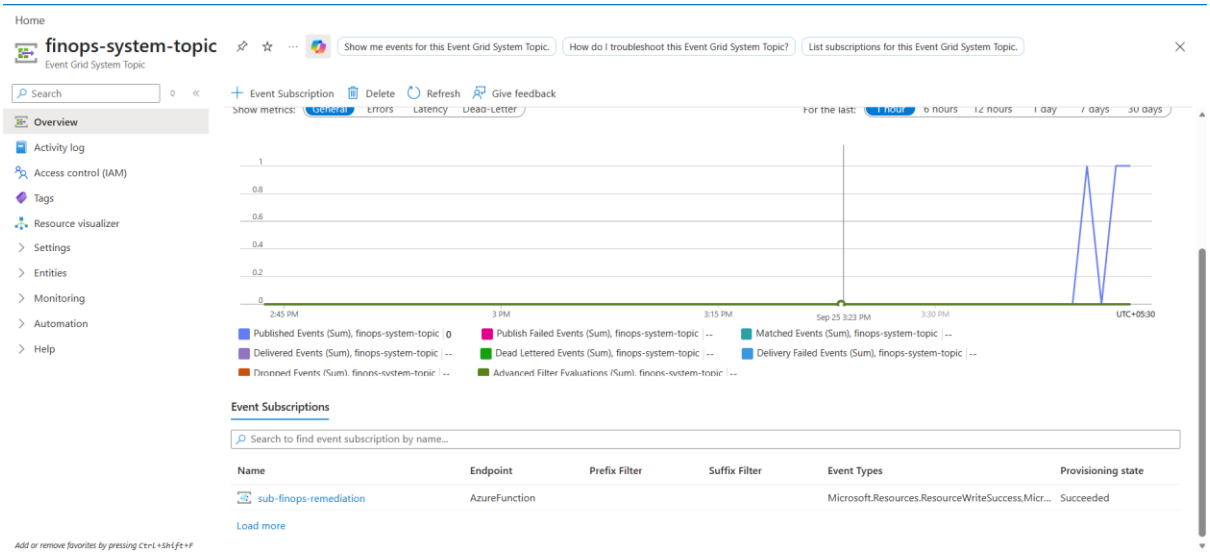


Figure 15: Topic overview: subscription sub-finops-remediation Succeeded, with event metrics

Phase 7: Identities and permissions (RBAC)

Purpose: *Give each identity only the permissions it needs.*

Identity	Role	Scope	Why
Function App (managed identity)	Tag Contributor	Resource group	Apply the Department tag.
Function App (managed identity)	Public IP Deleter (FinOps), custom	Resource group	Delete public IPs only (Phase 12).
Function App (managed identity)	Cosmos DB Built-in Data Contributor	Cosmos account	Write audit records (Phase 10).
Function App (managed identity)	Key Vault Secrets User	Key Vault	Read the Logic App URL secret (Phase 9).
Administrator account	Key Vault Secrets Officer	Key Vault	Create the secret.
Dashboard web app (managed identity)	Cosmos DB Built-in Data Reader	Cosmos account	Read records for the dashboard (Phase 16).
Function App (managed identity)	Contributor (removed)	Resource group	Used temporarily during testing, then removed to enforce least privilege.

Assign Tag Contributor

1. Resource group > **Access control (IAM)** > **Add** > **Add role assignment**.
2. Role: **Tag Contributor** > Next. Assign access to: **Managed identity** > **Select members** > Function App > `func-finops-orchestrator-shamnan`.
3. Click **Review + assign** twice. Wait a few minutes for the role to take effect.

Remove the broad Contributor role

1. Resource group > **Settings > Locks**: make sure there is **no lock on the resource group itself** (a lock there blocks role changes).

2. **Access control (IAM) > Role assignments**: select Contributor for the function identity > **Remove**.
3. Retest tagging: delete the Department tag on `shamnandemo`, send its event in Test/Run, and confirm the tag returns.

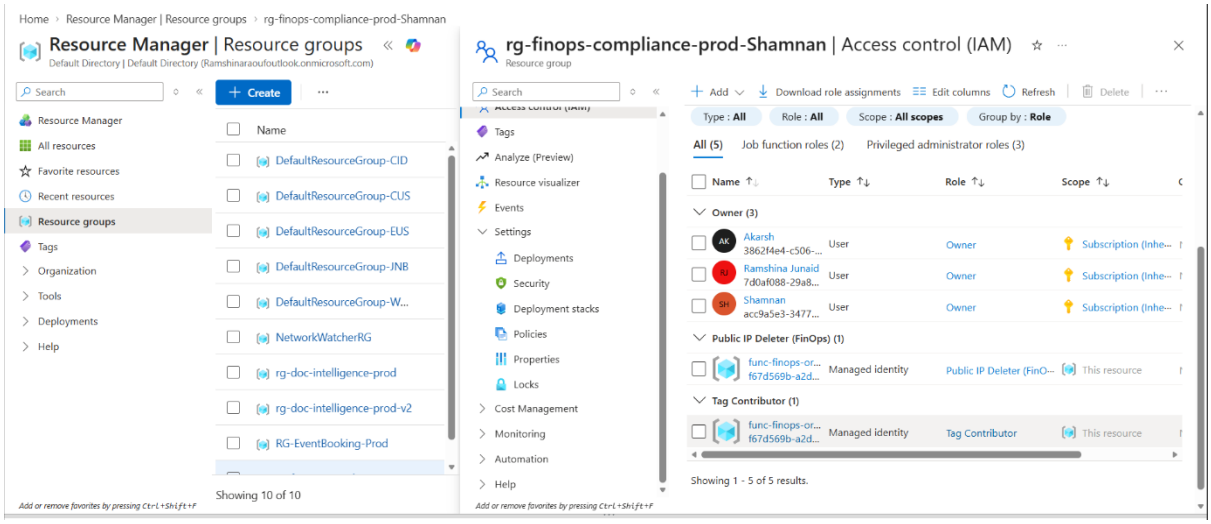


Figure 16: Role assignments list showing only Tag Contributor and Public IP Deleter (FinOps) for the function

Phase 8: Logic App approval workflow

Purpose: *Ask an administrator to approve or reject a high-risk resource, and wait for the answer.*

1. **Create a resource** > **Logic App**. On the hosting page choose **Consumption > Multi-tenant** (pay per run, near-zero idle cost). Do not choose a Standard plan.
2. Resource group: the project group. Name: `logic-finops-approval-shamnan`. Region: South India. Tags: `Department` = `IT`. Create > Go to resource > **Edit**.
3. Add trigger **When a HTTP request is received**. Under *Use sample payload to generate schema* paste the payload below.

```
{
  "resourceId":
"/subscriptions/xxx/resourceGroups/rg/providers/Microsoft.Network/publicIPAddresses/demo",
  "violation": "High-risk resource created",
  "proposedAction": "Review and approve",
  "eventTime": "2026-10-05T10:00:00Z"
}
```

1. Add action **Outlook.com > Send approval email** and sign in. To: your email. Subject: `Compliance approval needed`. User Options: `Approve, Reject`. Body (advanced parameter): the lines Violation, Resource, Proposed action and Time, each followed by its dynamic-content chip.
2. Add **Control > Condition**: `SelectedOption` is equal to `Approve`.
3. True branch: **Send an email (V2)** with subject `Remediation APPROVED` and the `resourceId` chip in the body.
4. False branch: **Send an email (V2)** with subject `Remediation REJECTED` and the `resourceId` chip in the body.
5. Click **Save** and confirm there are no errors.

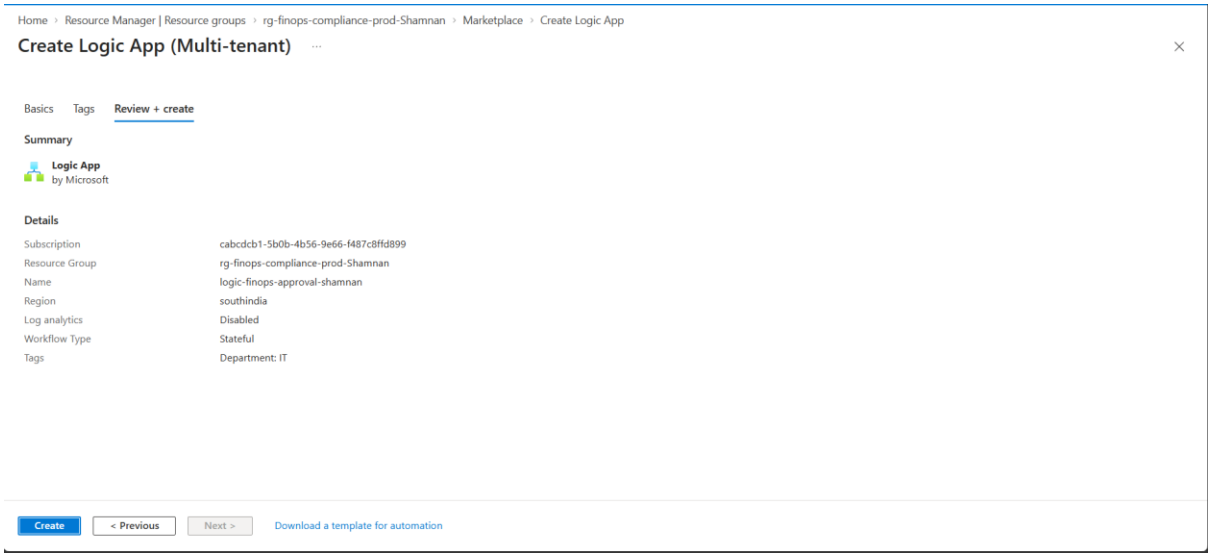


Figure 17: Hosting plan choice: Consumption, Multi-tenant

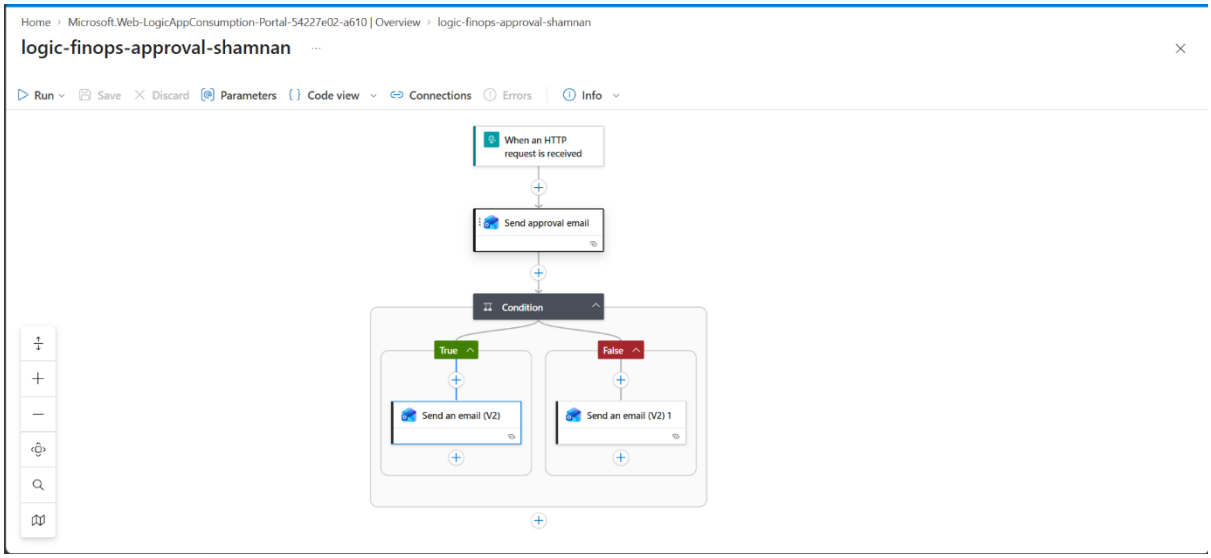


Figure 18: Logic App designer: trigger, approval email, condition, True and False branches

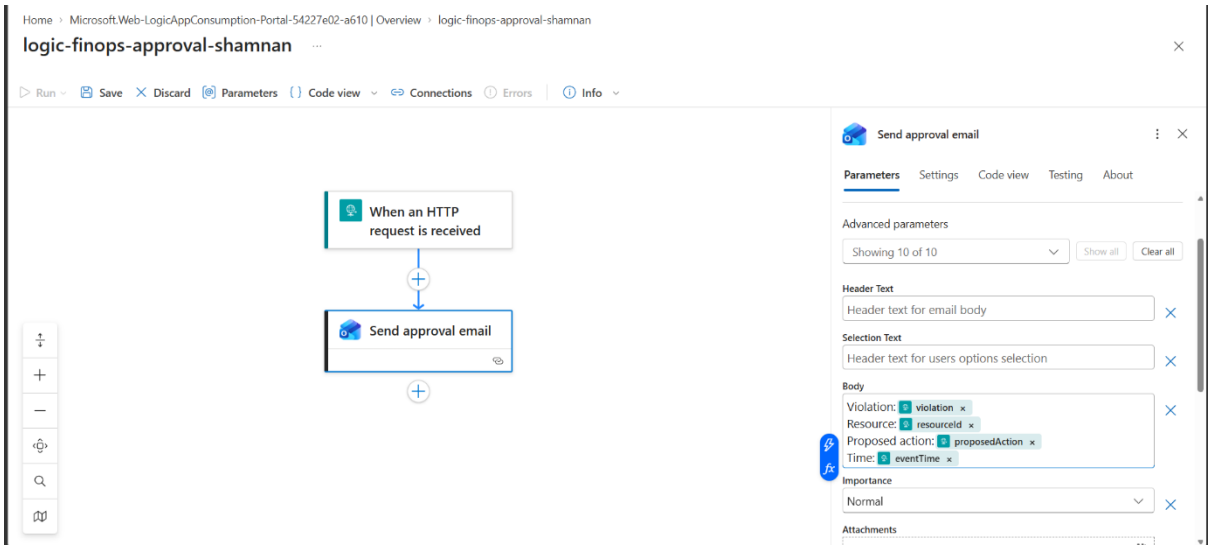


Figure 19: Send approval email settings (To, Subject, User Options, Body)

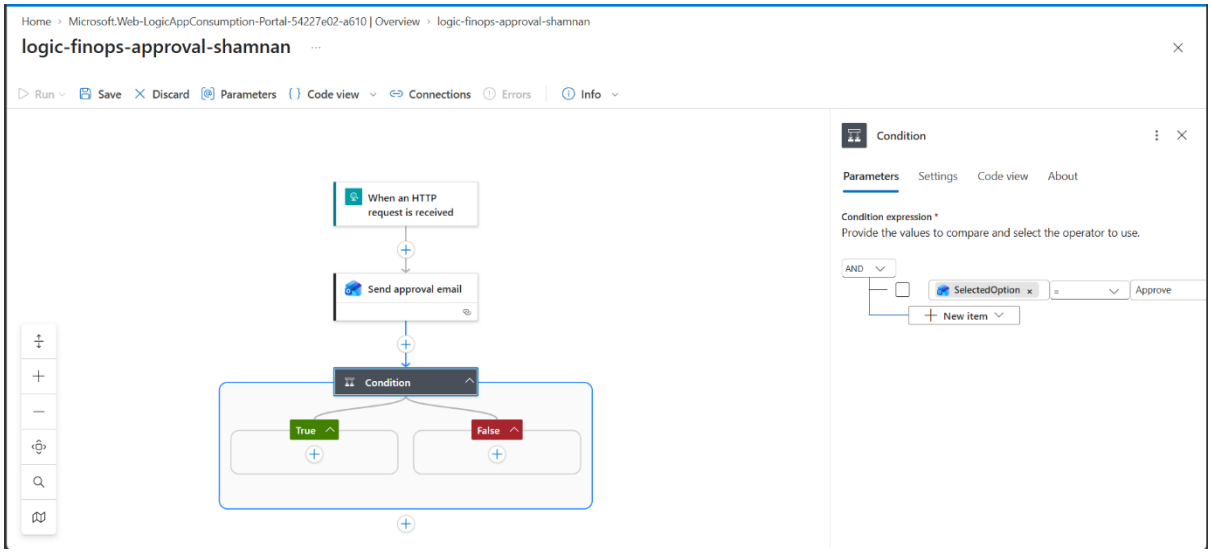


Figure 20: Condition: SelectedOption equals Approve

Phase 9: Store the Logic App URL in Key Vault

Purpose: *Keep the trigger URL secret and let the function read it without any password.*

1. Key Vault > **Access control (IAM)** > add **Key Vault Secrets Officer** to your own account. Wait a few minutes.
2. Logic App > **Settings > Access keys** > **Regenerate** (Primary key). Then open the designer, click the trigger and copy the new **HTTP URL**. Do not share it.
3. Key Vault > **Objects > Secrets > Generate/Import**. Name: `LogicAppApprovalUrl`. Secret value: the URL. Create.
4. Key Vault > **Access control (IAM)** > add **Key Vault Secrets User** to the Function App managed identity.
5. Function App > **Settings > Environment variables > App settings**: add or edit `LOGICAPP_URL` with the value below. Apply, then Apply and Confirm at the bottom.

```
@Microsoft.KeyVault(SecretUri=https://vault-finops-
shamnan.vault.azure.net/secrets/LogicAppApprovalUrl/)
```

1. Wait a minute and refresh. The setting should show a resolved Key Vault reference. Test with a VM event: the approval email must still arrive.

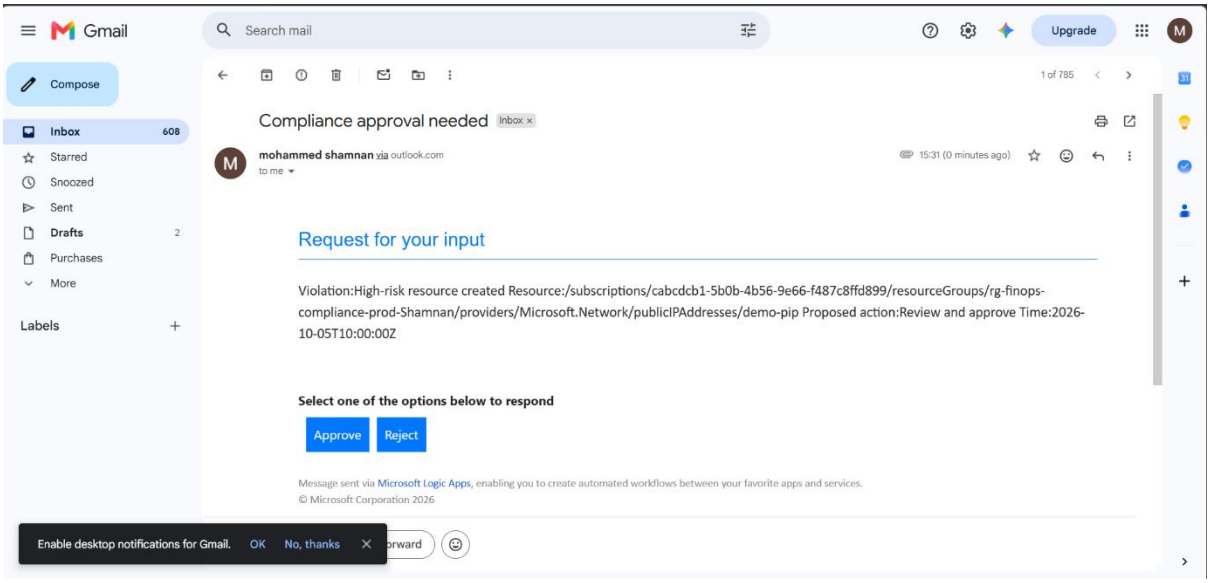


Figure 21: Key Vault secrets list showing LogicAppApprovalUrl (value hidden)

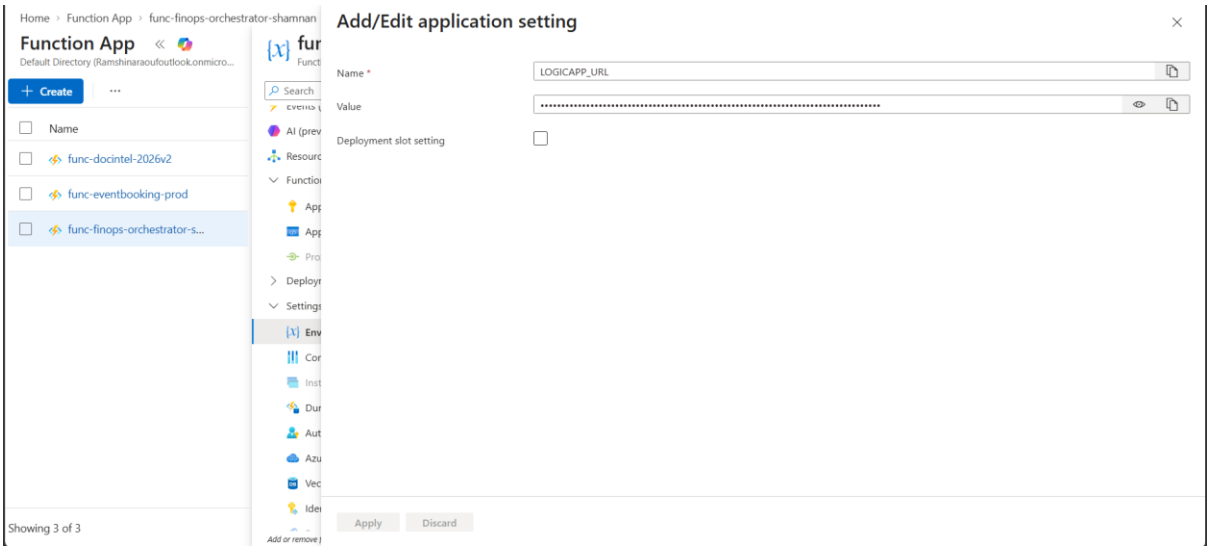


Figure 22: App setting LOGICAPP_URL showing a resolved Key Vault reference

> **Security:** a URL that has been shared anywhere (chat, email, screenshot) must be treated as exposed. Regenerate the Logic App access key and update the secret.

Phase 10: Audit logging to Cosmos DB

Purpose: *Record every detection and action so that there is a permanent history.*

1. Open Cloud Shell (**PowerShell**) and run the commands below to let the function write to Cosmos DB.

```
$principal = az functionapp identity show -n func-finops-orchestrator-shamnan -g rg-finops-compliance-prod-Shamnan --query principalId -o tsv

az cosmosdb sql role assignment create --account-name cosmos-finops-audit-shamnan --resource-group rg-finops-compliance-prod-Shamnan --scope "/" --principal-id $principal --role-definition-id 00000000-0000-0000-0000-000000000002
```

1. Function App > Environment variables: add the three settings in the table below. Apply, then Apply and Confirm.

Setting	Value
COSMOS_ENDPOINT	https://cosmos-finops-audit-shamnan.documents.azure.com
COSMOS_DB	ComplianceDB
COSMOS_CONTAINER	AuditLogs

1. Paste the final `run.ps1` (Appendix A) into **EventGridTrigger1 > Code + Test** and **Save**. Its `Write-AuditLog` function authenticates with the managed identity and posts a JSON record to the container.
2. Run a test and open **Cosmos DB > Data Explorer > ComplianceDB > AuditLogs > Items** to see the records.

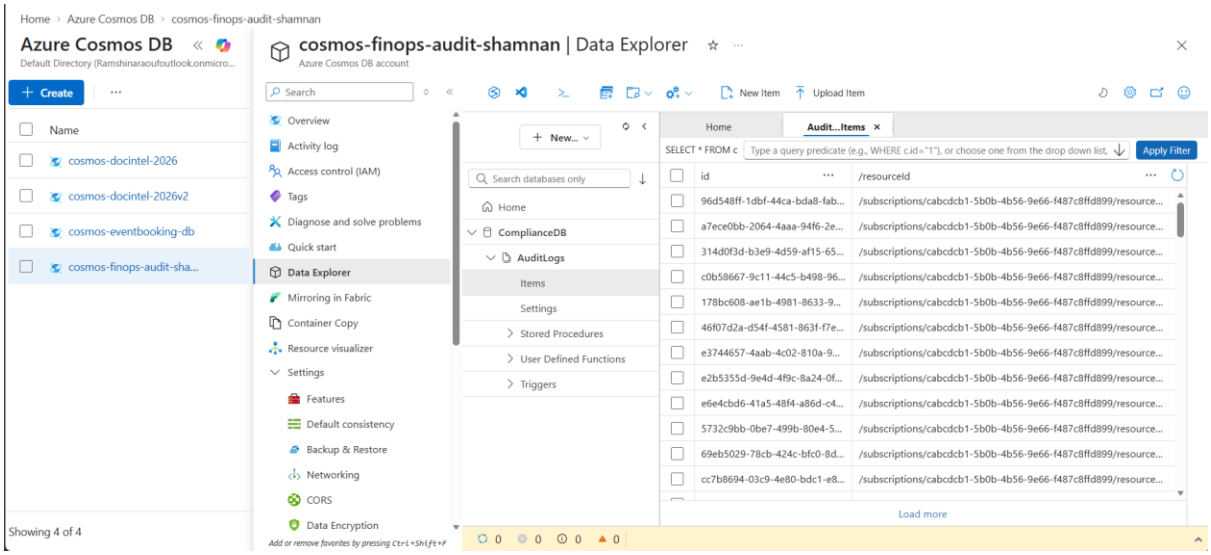


Figure 23: Cosmos DB Items list with audit records

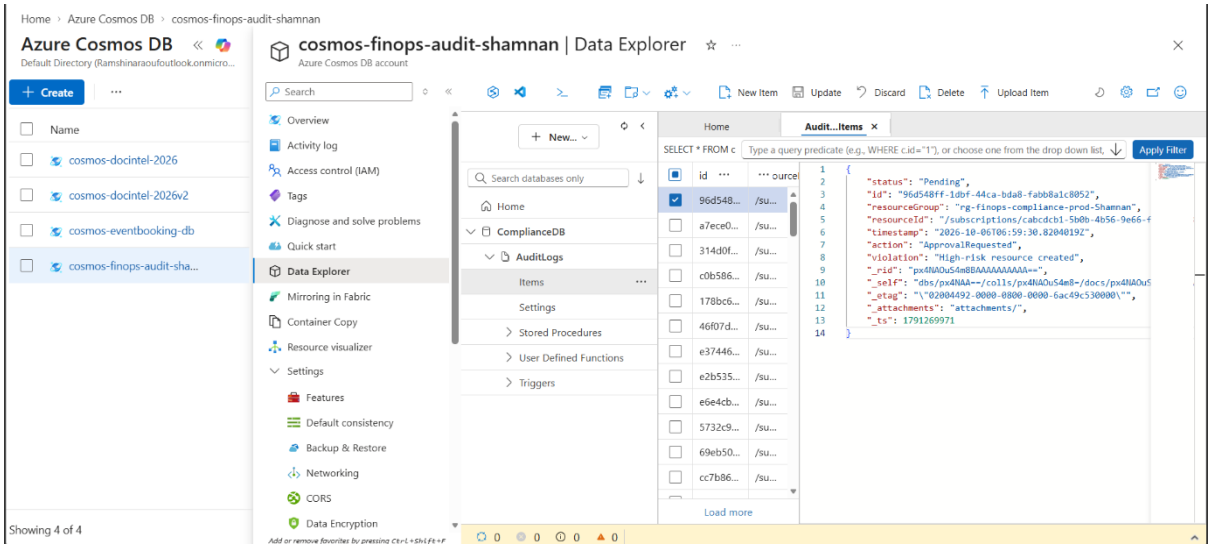


Figure 24: One expanded audit record (resourceId, violation, action, status, timestamp)

Phase 11: Function logic in Code + Test

Purpose: *Show the code that implements the decision logic from section 2.3.*

The complete script is in Appendix A. It contains the audit helper, the scope guards, the automatic-delete path, the approval path and the tagging path, in that order. Open **Functions > EventGridTrigger1 > Code + Test** to view it.

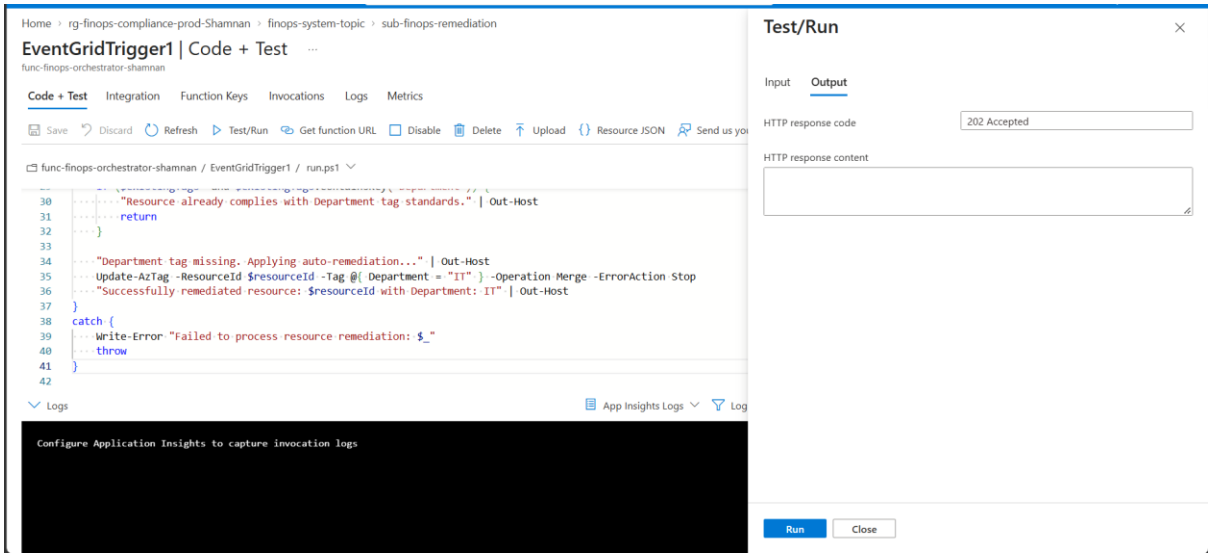


Figure 25: Code + Test showing run.ps1 with the Write-AuditLog function and decision logic

Phase 12: Automatic delete option

Purpose: *Remove unapproved public IP addresses automatically, safely, and only when the owner has switched the feature on.*

12.1 Why it is built this way

Deleting a resource cannot be undone, so the feature is surrounded by several independent safeguards. A single mistake cannot cause damage because every layer would have to fail at the same time.

Safeguard	What it does
Mode switch	AUTO_DELETE_MODE is off by default. Only the exact values dryrun and on activate the feature. Any other value falls back to the normal approval email.
Dry-run mode	Logs 'would delete' and writes a WouldDelete record, but deletes nothing. Used to validate the feature.
Keep tag	A public IP tagged Keep = true is never deleted. The skip is logged as DeleteSkipped.
Scope guard	Only resources inside the project resource group are ever processed.
Resource-type limit	Only public IP addresses are deleted automatically. Virtual machines always go to human approval.
Narrow custom role	The function can only read and delete public IPs. It cannot delete a Key Vault, Cosmos DB, storage account or the Function App itself.
Resource locks	Delete locks on the critical resources block deletion even by mistake.
Audit trail	Every outcome is written to Cosmos DB.
Kill switch	Set AUTO_DELETE_MODE to off, or remove the role from the function.

12.2 Operating modes

AUTO_DELETE_MODE	Public IP created	Virtual machine created
off (or not set)	Approval email	Approval email
dryrun	Logged as WouldDelete, nothing deleted	Approval email
on	Deleted automatically, unless Keep = true	Approval email

12.3 Create the narrow role

1. In Cloud Shell (**PowerShell**) run the following. Replace the subscription ID with your own.

```
$sub = "<subscription-id>"

@"
{
  "Name": "Public IP Deleter (FinOps)",
  "IsCustom": true,
  "Description": "Read and delete public IP addresses only",
  "Actions": [
    "Microsoft.Network/publicIPAddresses/read",
    "Microsoft.Network/publicIPAddresses/delete"
  ],
  "NotActions": [],
  "AssignableScopes": [
    "/subscriptions/$sub/resourceGroups/rg-finops-compliance-prod-Shamnan"
  ]
}
"@ | Out-File role.json -Encoding utf8

az role definition create --role-definition role.json
```

1. Assign the role to the function identity:

```
$fn = az functionapp identity show -n func-finops-orchestrator-shamnan -g rg-finops-
compliance-prod-Shamnan --query principalId -o tsv

az role assignment create --assignee-object-id $fn --assignee-principal-type ServicePrincipal
--role "Public IP Deleter (FinOps)" --scope /subscriptions/$sub/resourceGroups/rg-finops-
compliance-prod-Shamnan
```

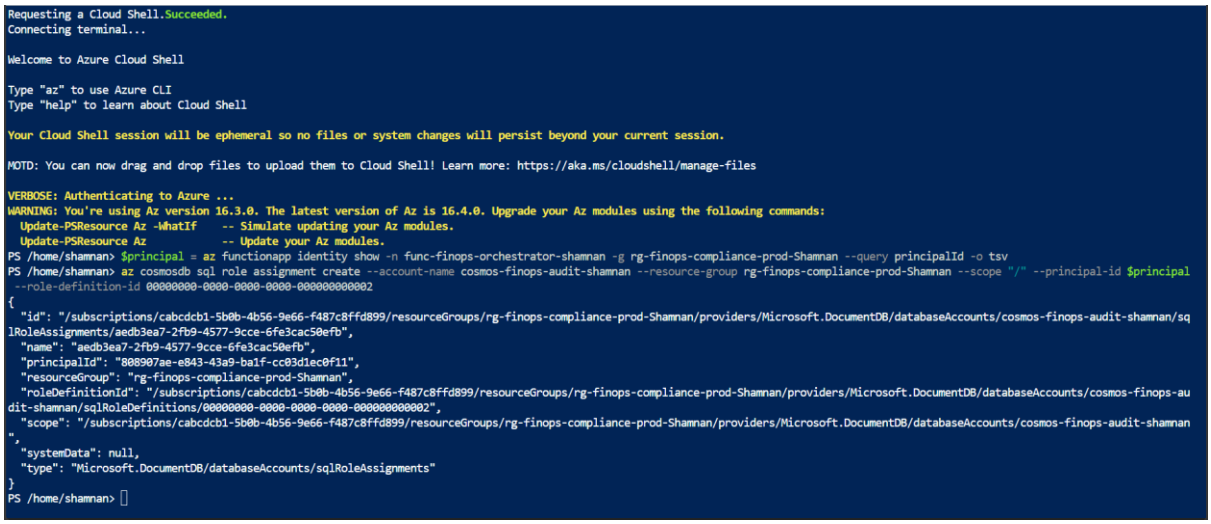


Figure 26: Cloud Shell output after creating the Public IP Deleter (FinOps) role

12.4 Enable the mode and code

1. Function App > **Environment variables**: add `AUTO_DELETE_MODE` with value `dryrun`. Apply, then Apply and Confirm.
2. The code (Appendix A) activates the delete path only when the mode is dryrun or on. The condition is:

```
if ($resourceId -match "/providers/Microsoft.Network/publicIPAddresses/[^/]+$" -and ($mode -eq "dryrun" -or $mode -eq "on")) {
```

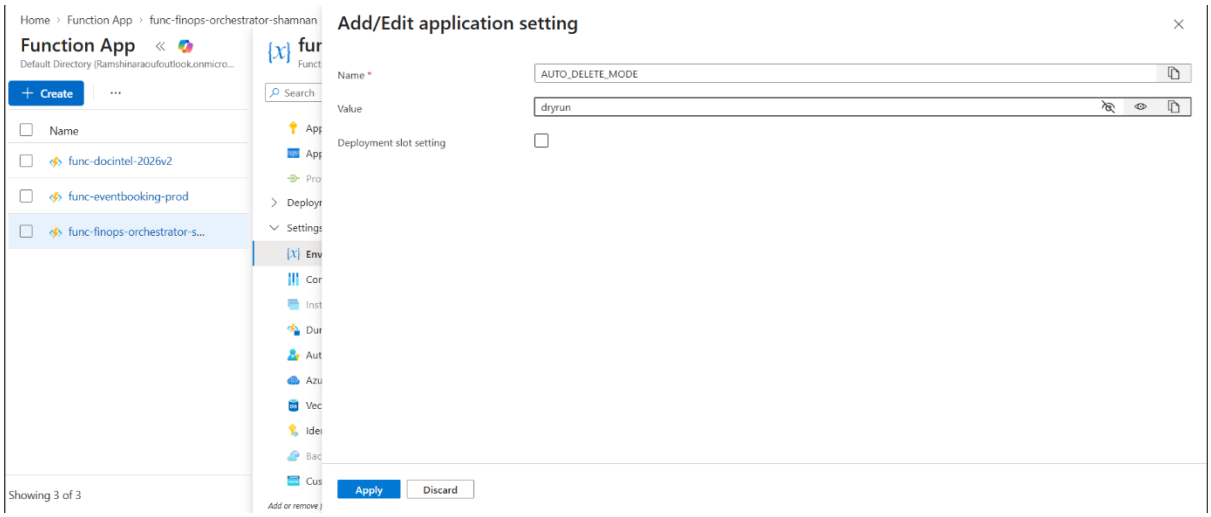


Figure 27: App setting AUTO_DELETE_MODE = dryrun

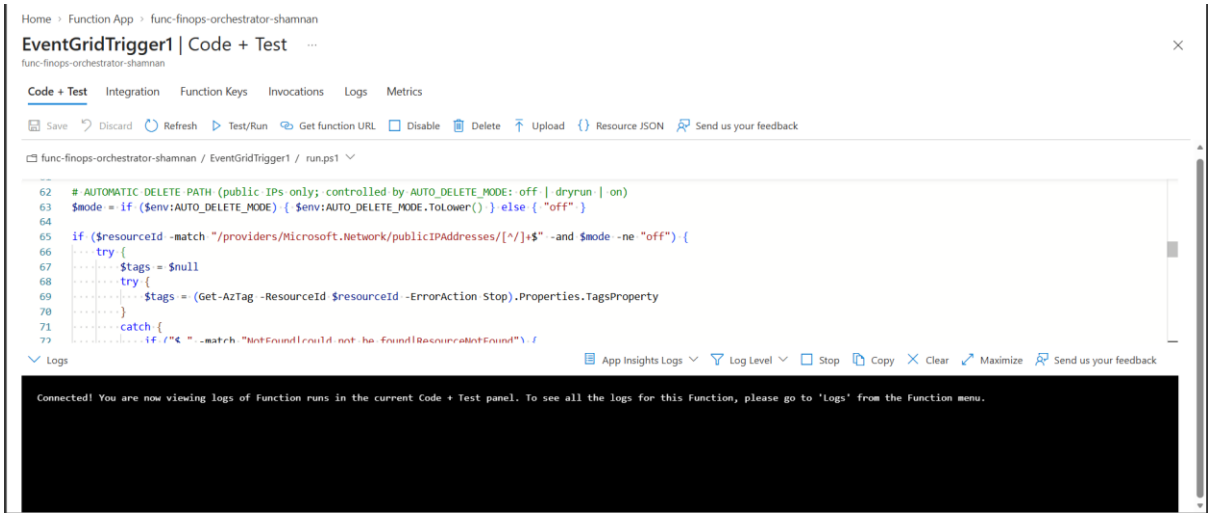


Figure 28: run.ps1 showing the AUTOMATIC DELETE PATH block

12.5 Test the feature in order

1. **Test A, dry run.** With the mode set to `dryrun`, create a public IP named `pip-auto-test-01` (SKU Standard, Static) in the project resource group. Expected: the log shows `DRY RUN: would delete`, the public IP **remains**, and Cosmos DB shows a record with action `WouldDelete` and status `DryRun`.
2. **Test B, Keep tag.** Create `pip-auto-keep-02` with the tag `Keep = true`. Expected: the log shows `Keep=true tag present`, the IP remains, and Cosmos DB shows `DeleteSkipped` with status `KeepTag`.
3. **Test C, real deletion.** Change the mode to `on` (Apply, then Apply and Confirm). Create another public IP without the Keep tag. Expected: it disappears from the resource group

within about a minute, the log shows `Deleted`, and Cosmos DB shows `AutoDeleted` with status `Success`.

4. **Test D, virtual machine.** Send the sample VM event. Expected: an approval email, never a deletion.
5. Afterwards delete the remaining test IPs manually (Standard public IPs bill by the hour) and set the mode back to `off` or `dryrun`.

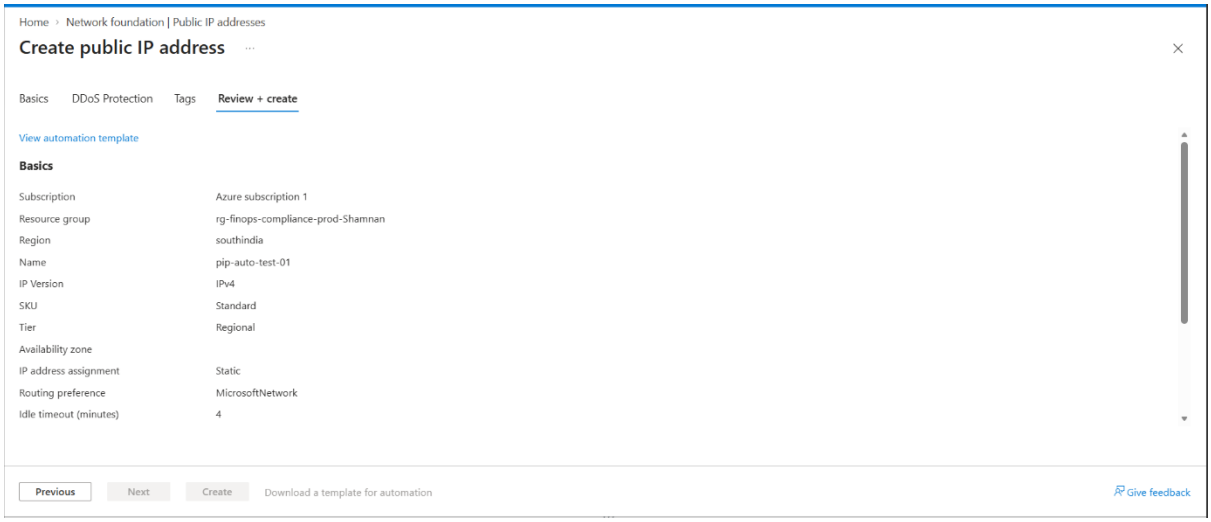


Figure 29: Test A: public IP created (review page) and still present after the dry run

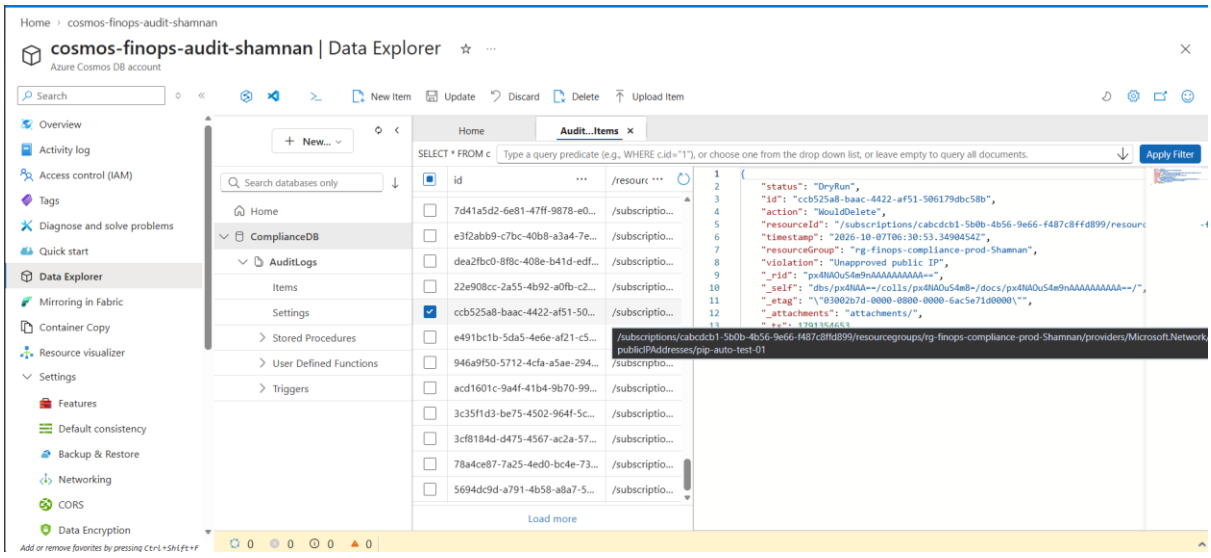


Figure 30: Test A: Cosmos DB record action WouldDelete, status DryRun

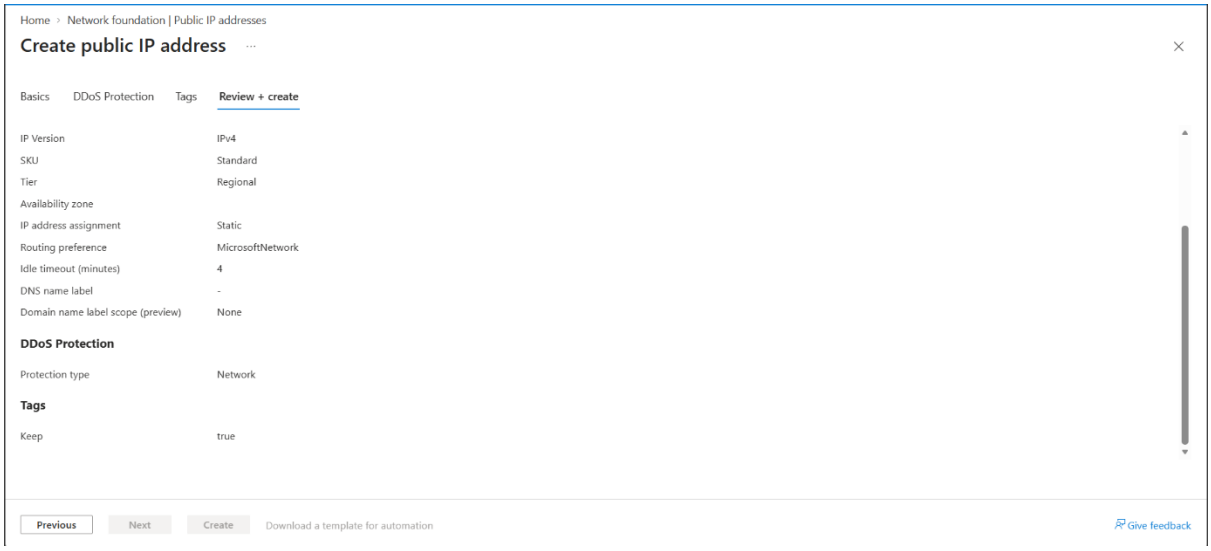


Figure 31: Test B: public IP with Keep = true tag (review page)

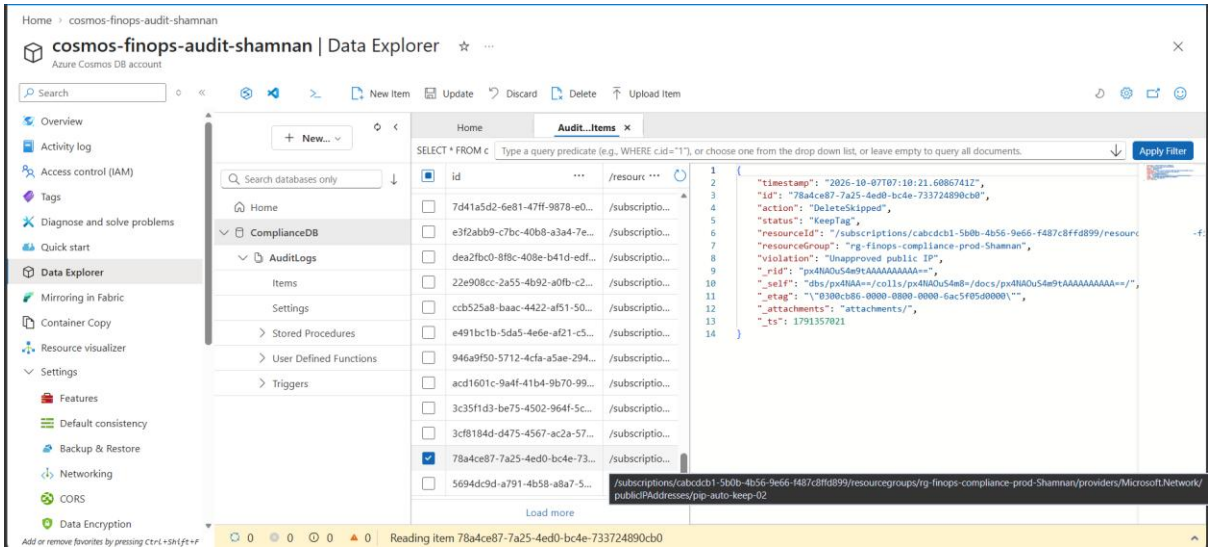


Figure 32: Test B: Cosmos DB record action DeleteSkipped, status KeepTag

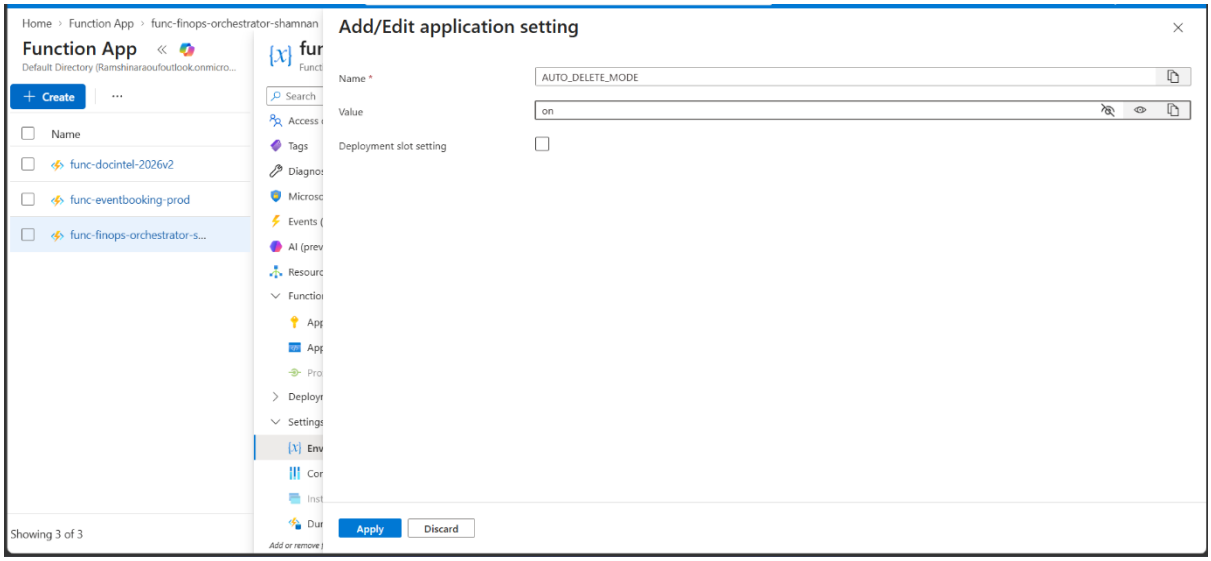


Figure 33: App setting AUTO_DELETE_MODE = on

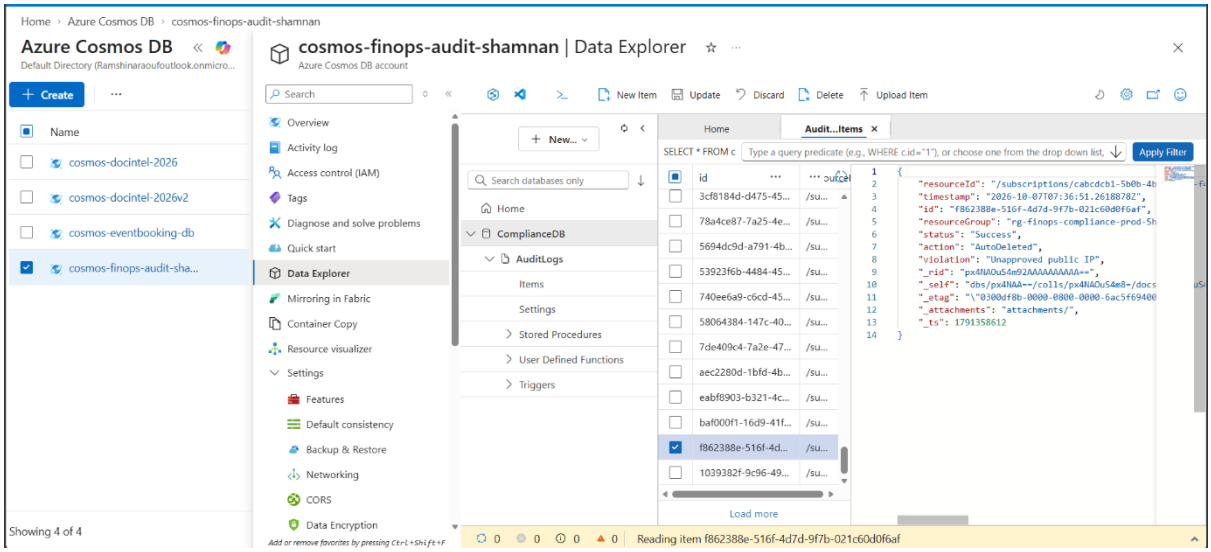


Figure 34: Test C: Cosmos DB record action AutoDeleted, status Success

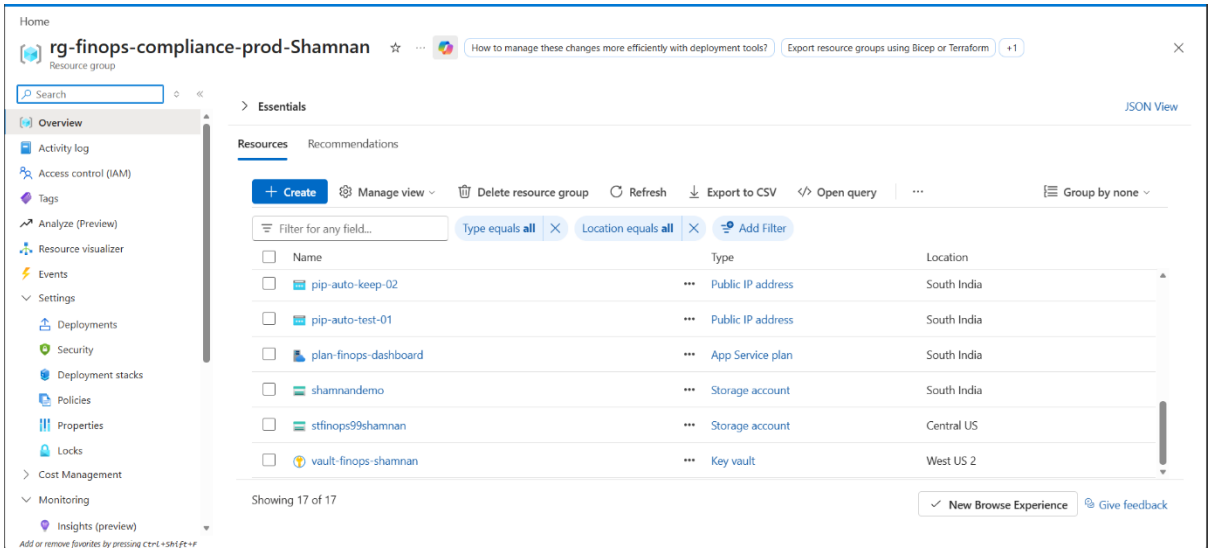


Figure 35: Resource group list after the tests (deleted IP is gone; protected IP remains)

> **Important:** when AUTO_DELETE_MODE is on, any public IP created in the project resource group is deleted unless it carries `Keep = true`. Leave the mode `off` between demonstrations. With the mode `dryrun` or `on`, a fake public IP event no longer sends an approval email, so use the virtual machine test event for the approval demo.

Phase 13: Monitoring and alerts (Azure Monitor)

Purpose: *Be notified by email whenever a remediation run fails.*

1. Function App > **Application Insights** > **View Application Insights data** > **Alerts** > **Create** > **Alert rule**.
2. Condition: **Custom log search** with the query below. Alert logic: Table rows, Count, greater than 0, check every 5 minutes, lookback 5 minutes.

```
requests
| where cloud_RoleName =~ "func-finops-orchestrator-shamnan"
| where success == false
```

1. Actions: **Create action group**, name `ag-finops-alerts`. Notification type **Email**, with your email address (check the spelling).

2. Details: rule name `alert-finops-failed-remediation`, severity 2 (Warning). Review + create.
3. Test: send an event for a storage account that does not exist (Appendix B). The function fails on purpose and the alert email arrives after a few minutes.

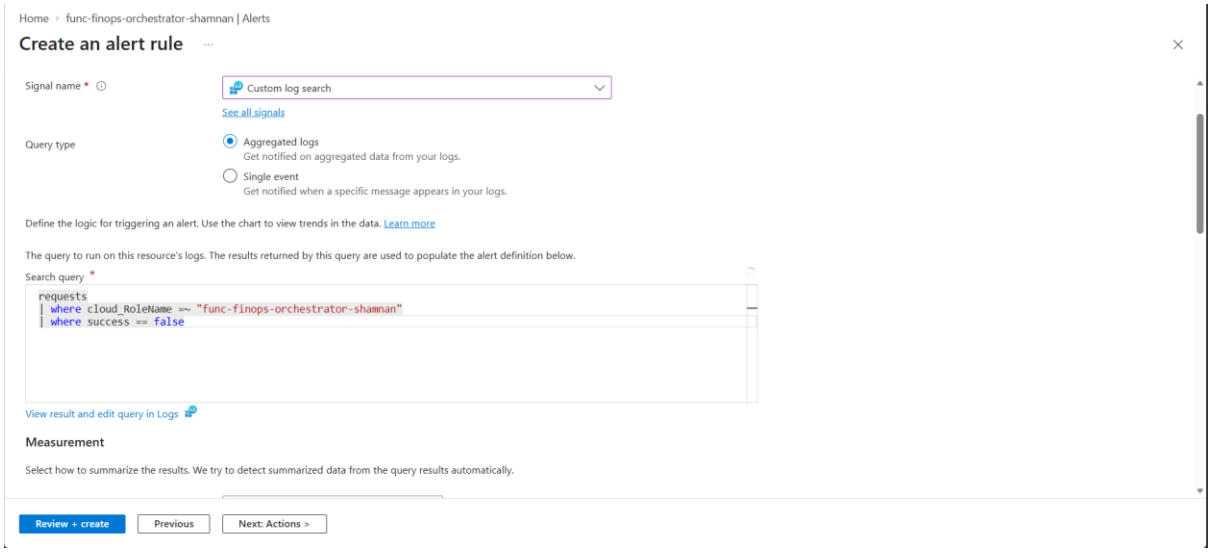


Figure 36: Alert rule configuration (query and threshold)

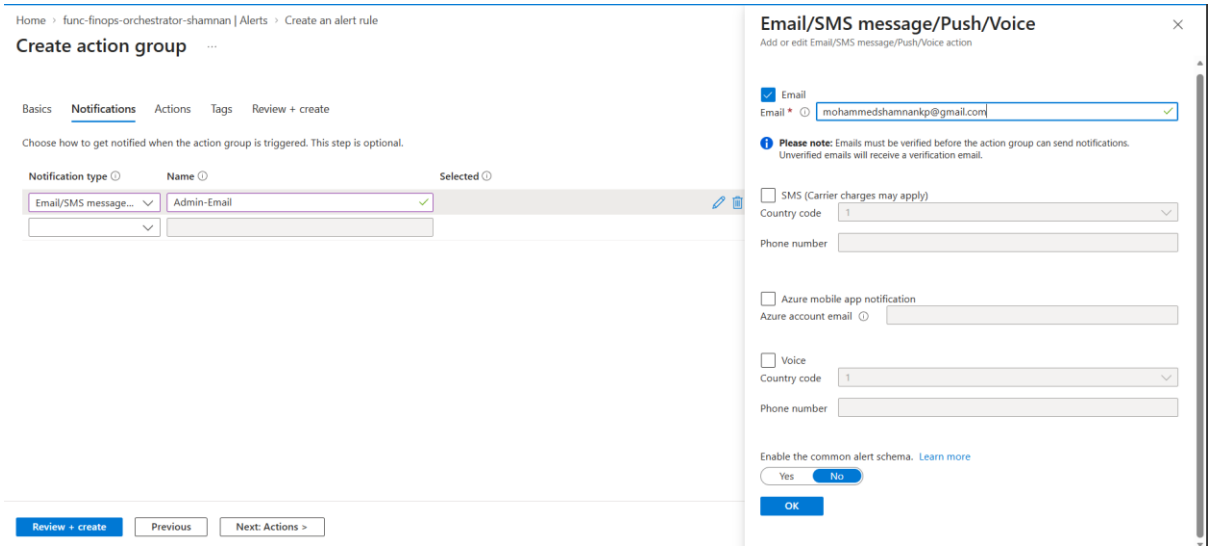


Figure 37: Action group with the email notification

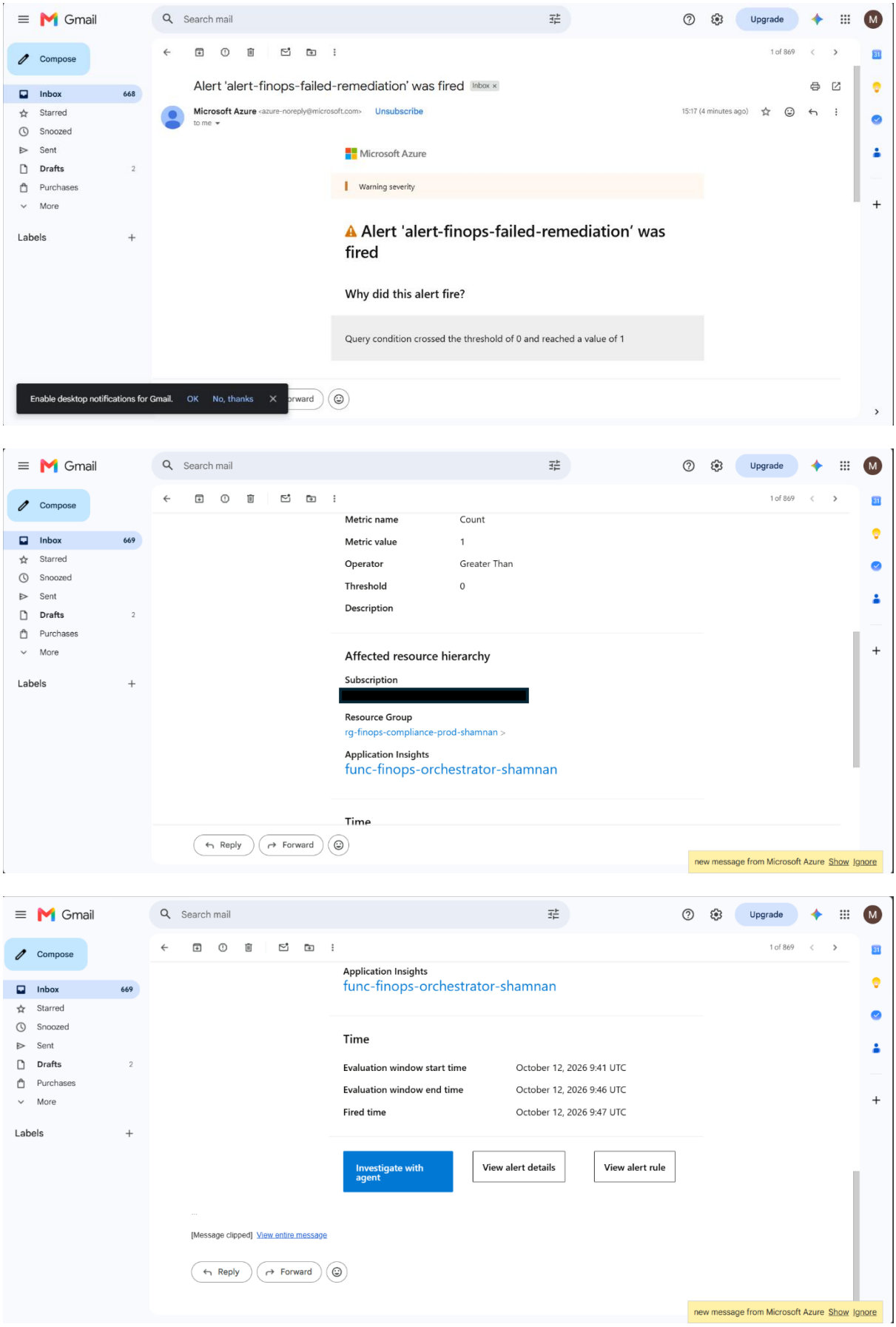


Figure 38: Alert email received after the deliberate failure

Phase 14: Microsoft Defender for Cloud

Purpose: *Assess the security posture of the subscription.*

1. Portal search > **Microsoft Defender for Cloud** > **Environment settings** > select *Azure subscription 1*.
2. On **Defender plans**, make sure **Foundational CSPM** is **On** (free). Leave every paid plan **Off** to avoid charges. Save.
3. Open **Overview** and note the **Secure score**, then open **Recommendations** and select one relevant recommendation. The score can take up to a day to appear.

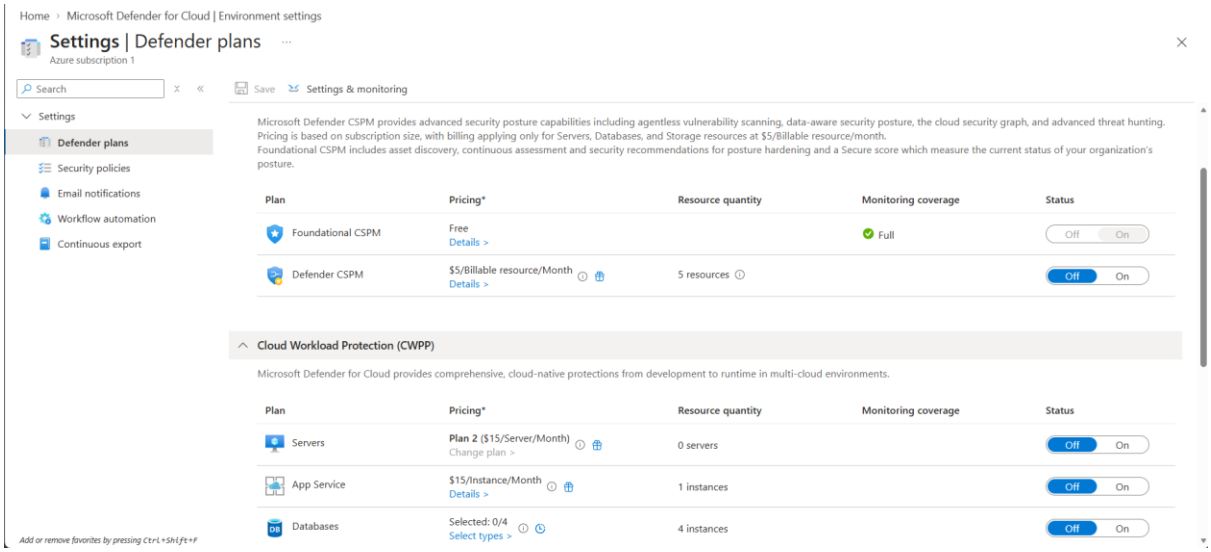


Figure 39: Defender plans page with Foundational CSPM On and paid plans Off

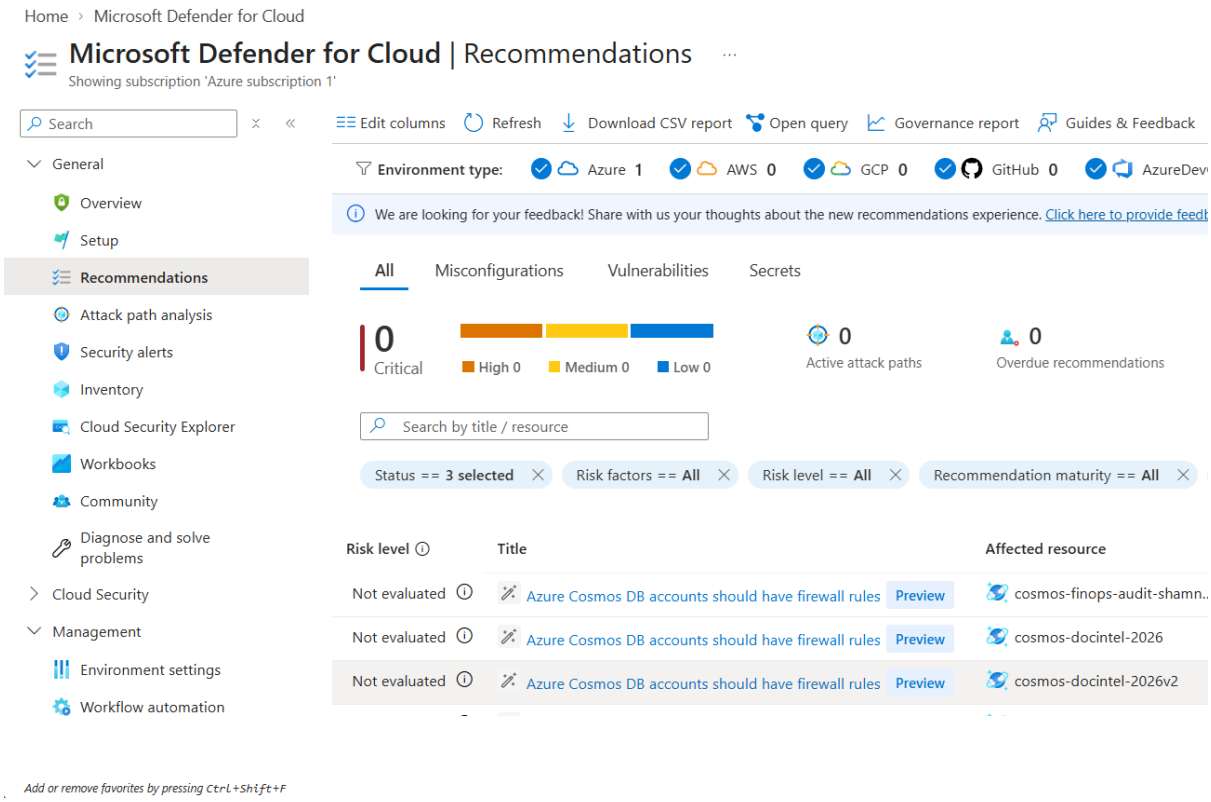


Figure 40: Secure score and one recommendation

Phase 15: Azure Container Instances vulnerability scan

Purpose: Run a periodic batch security scan as a short-lived container.

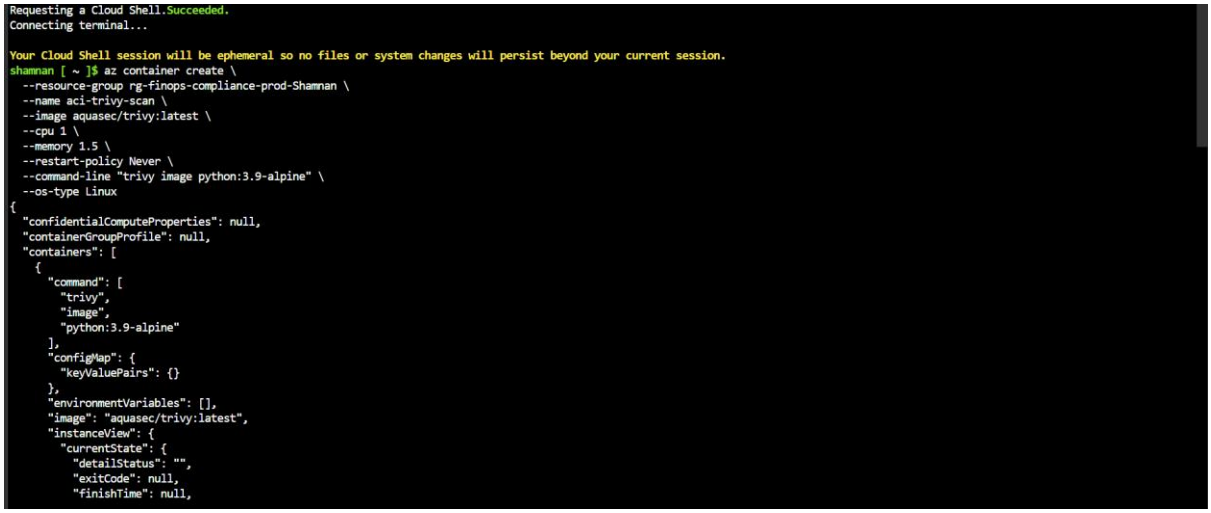
1. Open **Azure Cloud Shell** (Bash) and create the container group with the Azure CLI. The scan uses the Trivy image and runs once:

```
az container create \
 --resource-group rg-finops-compliance-prod-Shamnan \
 --name aci-trivy-scan \
 --image aquasec/trivy:latest \
 --cpu 1 \
 --memory 1.5 \
 --restart-policy Never \
 --command-line "trivy image python:3.9-alpine" \
 --os-type Linux
```

2. The restart policy `Never` makes the container run once and stop. The output shows the container state change to **Running**, with the image pulled and the container started.
3. When the scan finishes, read the result with:

```
az container logs \
 --resource-group rg-finops-compliance-prod-Shamnan \
 --name aci-trivy-scan
```

4. The scan result lists the vulnerabilities found in the `python:3.9-alpine` image, with severity, installed version and fixed version. In this run it reported 11 Python package findings (1 low, 7 medium, 3 high, 0 critical).



```
      "detailStatus": "",
      "exitCode": null,
      "finishTime": null,
      "startTime": "2026-10-06T08:57:10.224000+00:00",
      "state": "Running"
    },
    "events": [
      {
        "count": 1,
        "firstTimestamp": "2026-10-06T08:56:40+00:00",
        "lastTimestamp": "2026-10-06T08:56:40+00:00",
        "message": "pulling image \"aquasec/trivy@sha256:9db099105405c648166e6b94155eb32f8da12673cf1f455207f7385cc9a77283\"",
        "name": "Pulling",
        "type": "Normal"
      },
      {
        "count": 1,
        "firstTimestamp": "2026-10-06T08:56:52+00:00",
        "lastTimestamp": "2026-10-06T08:56:52+00:00",
        "message": "Successfully pulled image \"aquasec/trivy@sha256:9db099105405c648166e6b94155eb32f8da12673cf1f455207f7385cc9a77283\"",
        "name": "Pulled",
        "type": "Normal"
      },
      {
        "count": 1,
        "firstTimestamp": "2026-10-06T08:57:10+00:00",
        "lastTimestamp": "2026-10-06T08:57:10+00:00",
        "message": "Started container",
        "name": "Started",
        "type": "Normal"
      }
    ],
    "previousState": null,
```


Figure 41: Container group overview

```
shamnan [ ~ ]$ az container logs \
  --resource-group rg-finops-compliance-prod-Shamnan \
  --name aci-trivy-scan
```

ssl_client	CVE-2024-58251			1.37.0-r19	1.37.0-r20	In netstat in BusyBox through 1.37.0, local users can launch of networ... https://avd.aquasec.com/nvd/cve-2024-58251
	CVE-2025-46394	LOW				In tar in BusyBox through 1.37.0, a TAR archive can have filenames... https://avd.aquasec.com/nvd/cve-2025-46394
xz-libs	CVE-2026-34743	MEDIUM		5.8.1-r0	5.8.3-r0	xz: XZ Utils: Denial of Service via buffer overflow in index decoding... https://avd.aquasec.com/nvd/cve-2026-34743
zlib	CVE-2026-22184	HIGH		1.3.1-r2	1.3.2-r0	zlib: zlib: Arbitrary code execution via buffer overflow in untgz utility https://avd.aquasec.com/nvd/cve-2026-22184
	CVE-2026-27171	MEDIUM				zlib: zlib: Denial of Service via infinite loop in CRC32 combine functions... https://avd.aquasec.com/nvd/cve-2026-27171

```
Python (python-pkg)
==================
Total: 11 (UNKNOWN: 0, LOW: 1, MEDIUM: 7, HIGH: 3, CRITICAL: 0)
```

Library	Vulnerability	Severity	Status	Installed Version	Fixed Version	Title
jaraco.context (METADATA)	CVE-2026-23949	HIGH	fixed	5.3.0	6.1.0	jaraco.context: jaraco.context: Path traversal via malicious tar archives



Python (python-pkg)
==========
Total: 11 (UNKNOWN: 0, LOW: 1, MEDIUM: 7, HIGH: 3, CRITICAL: 0)

Library	Vulnerability	Severity	Status	Installed Version	Fixed Version	Title
jaraco.context (METADATA)	CVE-2026-23949	HIGH	fixed	5.3.0	6.1.0	jaraco.context: jaraco.context: Path traversal via malicious tar archives https://avd.aquasec.com/nvd/cve-2026-23949
pip (METADATA)	CVE-2023-5752	MEDIUM		23.0.1	23.3	pip: Mercurial configuration injectable in repo revision when installing via pip https://avd.aquasec.com/nvd/cve-2023-5752
	CVE-2025-8869				25.3	pip: pip missing checks on symbolic link extraction https://avd.aquasec.com/nvd/cve-2025-8869
	CVE-2026-13346				26.2.0	pip: pip: Arbitrary file installation via malicious package indexes https://avd.aquasec.com/nvd/cve-2026-13346
	CVE-2026-3219				26.1	pip: pip: Incorrect file installation due to improper archive handling https://avd.aquasec.com/nvd/cve-2026-3219
	CVE-2026-6357					pip: pip: Arbitrary code execution or information disclosure via malicious wheel package... https://avd.aquasec.com/nvd/cve-2026-6357
	CVE-2026-8643				26.1.2	python-pip: Path traversal via malicious entry point name in pip wheel installation... https://avd.aquasec.com/nvd/cve-2026-8643
						indexes https://avd.aquasec.com/nvd/cve-2026-13346
	CVE-2026-3219				26.1	pip: pip: Incorrect file installation due to improper archive handling https://avd.aquasec.com/nvd/cve-2026-3219
	CVE-2026-6357					pip: pip: Arbitrary code execution or information disclosure via malicious wheel package... https://avd.aquasec.com/nvd/cve-2026-6357
	CVE-2026-8643				26.1.2	python-pip: Path traversal via malicious entry point name in pip wheel installation... https://avd.aquasec.com/nvd/cve-2026-8643
	CVE-2026-1703	LOW			26.0	pip: pip: Information disclosure via path traversal when installing crafted wheel archives... https://avd.aquasec.com/nvd/cve-2026-1703
setuptools (METADATA)	CVE-2026-59890	MEDIUM		79.0.1	83.0.0	setuptools: setuptools: MANIFEST.in exclusion bypass in sdist via Unicode normalization collision (NFC/NFD)... https://avd.aquasec.com/nvd/cve-2026-59890
wheel (METADATA)	CVE-2026-24049	HIGH		0.45.1	0.46.2	wheel: wheel: Privilege Escalation or Arbitrary Code Execution via malicious wheel file... https://avd.aquasec.com/nvd/cve-2026-24049

shamnan [~]$

Figure 42: Container logs showing the vulnerability scan result

Phase 16: Admin dashboard (Azure App Service)

Purpose: *Show the audit ledger as a web page for administrators.*

1. **Create a resource** > **Web App**. Resource group: the project group. Runtime stack: **Python 3.11**, OS **Linux**, pricing plan **Free F1** (plan `plan-finops-dashboard`).
2. Web App > **Settings > Identity > System assigned > On**.
3. In Cloud Shell give it read access to Cosmos DB (replace the web app name):

```
$web = az webapp identity show -n <web-app-name> -g rg-finops-compliance-prod-Shamnan --query principalId -o tsv

az cosmosdb sql role assignment create --account-name cosmos-finops-audit-shamnan --resource-group rg-finops-compliance-prod-Shamnan --scope "/" --principal-id $web --role-definition-id 00000000-0000-0000-0000-000000000001
```

1. Web App > Environment variables: add `COSMOS_ENDPOINT`, `COSMOS_DB`, `COSMOS_CONTAINER` (same values as in Phase 10) and `SCM_DO_BUILD_DURING_DEPLOYMENT = true`. Startup command: `gunicorn --bind=0.0.0.0 --timeout 600 app:app`.
2. Deploy the application code (reference implementation in Appendix C) as a zip package.
3. Open the web app URL. The page lists the audit records with totals per action.
4. Web App > **Networking** > restrict public access to your own IP address.

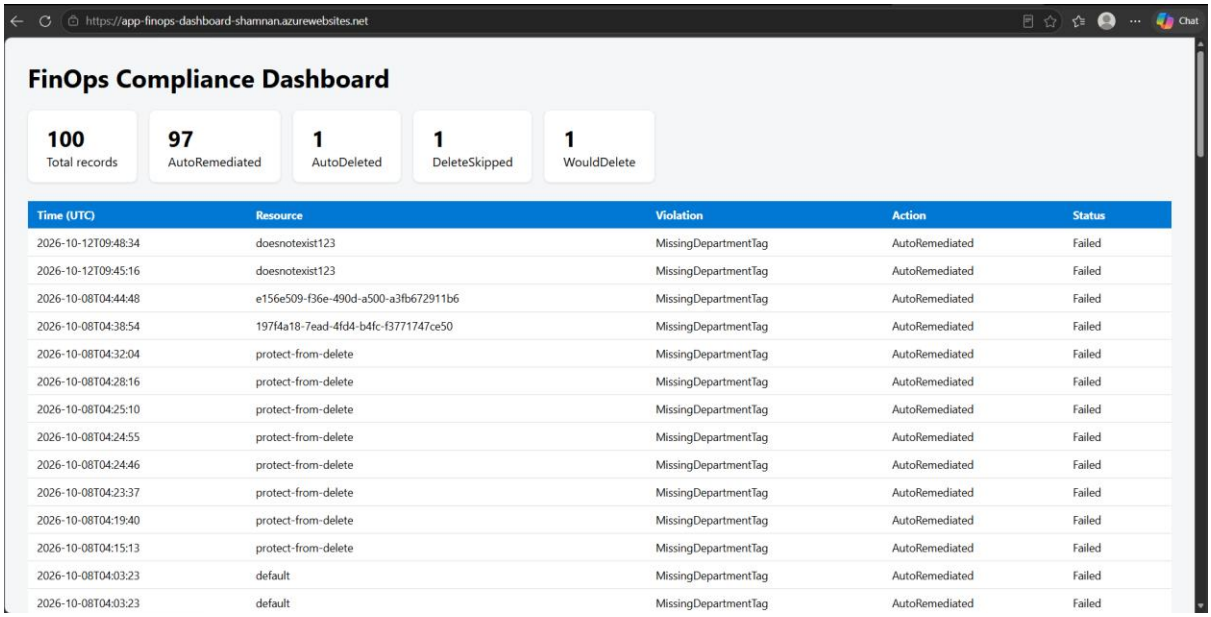


Figure 43: Dashboard in the browser listing audit records and totals

NOTE : The dashboard shows the 100 most recent audit records. Many entries are AutoRemediated with status Failed: these come from events for sub-resources that cannot carry tags (locks, role definitions, tags/default) and from the deliberate failure test (doesnotexist123). The Phase 12 test results appear as WouldDelete, DeleteSkipped and AutoDeleted.

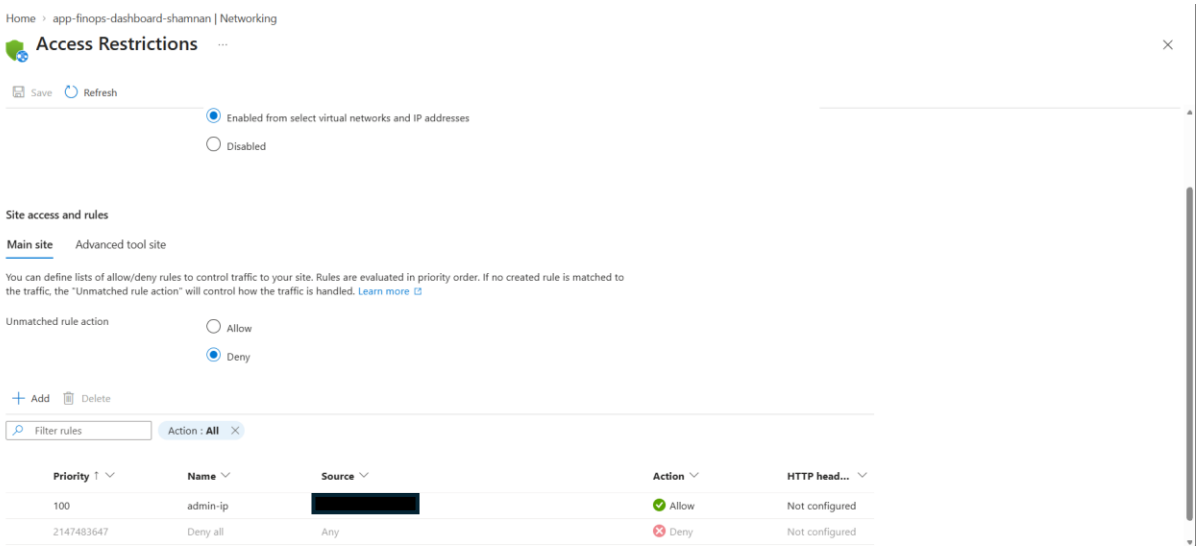


Figure 44: Web app networking restriction (access limited to the administrator IP)

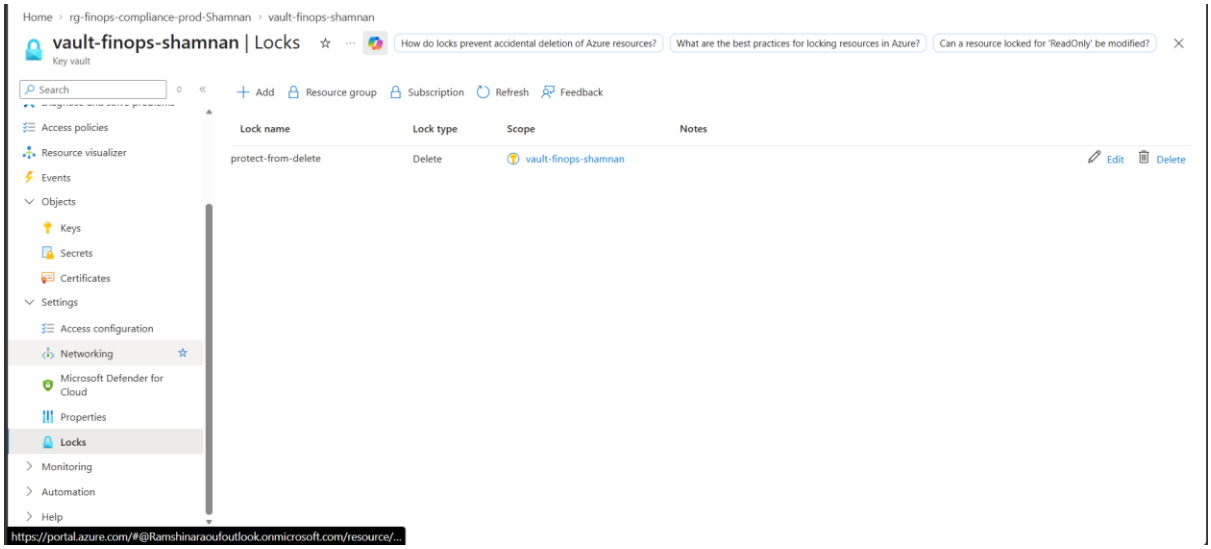
Phase 17: Resource locks

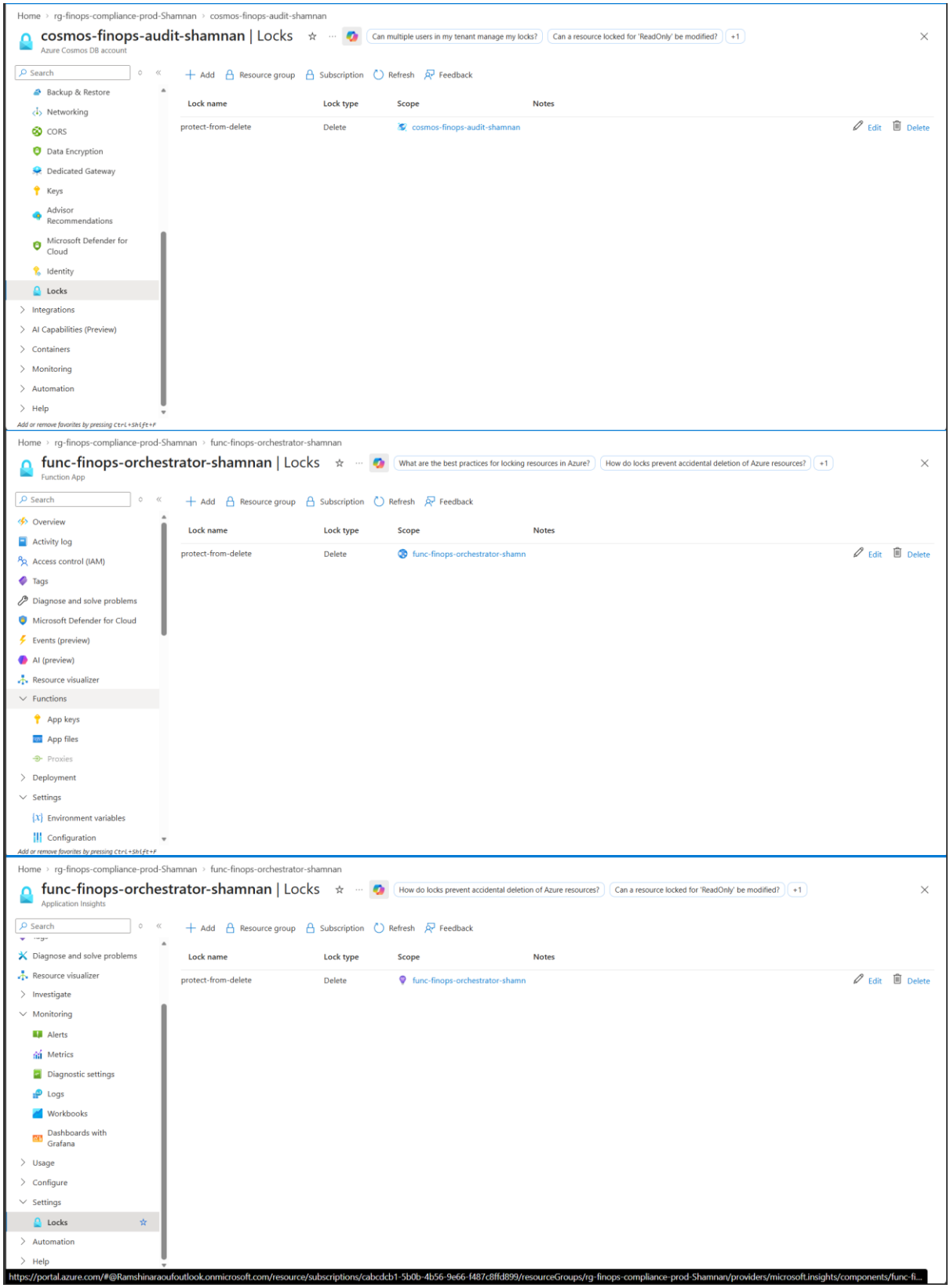
Purpose: *Prevent accidental deletion of the critical resources.*

1. For each resource in the table below: open the resource > **Settings > Locks** > **Add**. Name: `protect-from-delete`. Lock type: **Delete**. OK.
2. Do **not** lock the resource group itself, because that blocks role changes and the automatic deletion of public IPs.

Resource	Lock
vault-finops-shamnan	Delete

Resource	Lock
cosmos-finops-audit-shamnan	Delete
logic-finops-approval-shamnan	Delete
func-finops-orchestrator-shamnan (Function App) and its Application Insights	Delete
stfinops99shamnan	Delete
Resource group	None




Home › rg-finops-compliance-prod-Shamnan › cosmos-finops-audit-shamnan
cosmos-finops-audit-shamnan | Locks
Azure Cosmos DB account
Can multiple users in my tenant manage my locks?
Can a resource locked for 'ReadOnly' be modified?
+1
Search
Add
Resource group
Subscription
Refresh
Feedback
Backup & Restore
Networking
CORS
Data Encryption
Dedicated Gateway
Keys
Advisor Recommendations
Microsoft Defender for Cloud
Identity
Locks
Integrations
AI Capabilities (Preview)
Containers
Monitoring
Automation
Help
Add or remove favorites by pressing Ctrl+Shift+F
Lock name
Lock type
Scope
Notes
protect-from-delete
Delete
cosmos-finops-audit-shamnan
Edit
Delete
Home › rg-finops-compliance-prod-Shamnan › func-finops-orchestrator-shamnan
func-finops-orchestrator-shamnan | Locks
Function App
What are the best practices for locking resources in Azure?
How do locks prevent accidental deletion of Azure resources?
+1
Search
Add
Resource group
Subscription
Refresh
Feedback
Overview
Activity log
Access control (IAM)
Tags
Diagnose and solve problems
Microsoft Defender for Cloud
Events (preview)
AI (preview)
Resource visualizer
Functions
App keys
App files
Proxies
Deployment
Settings
Environment variables
Configuration
Add or remove favorites by pressing Ctrl+Shift+F
Lock name
Lock type
Scope
Notes
protect-from-delete
Delete
func-finops-orchestrator-shamn
Edit
Delete
Home › rg-finops-compliance-prod-Shamnan › func-finops-orchestrator-shamnan
func-finops-orchestrator-shamnan | Locks
Application Insights
How do locks prevent accidental deletion of Azure resources?
Can a resource locked for 'ReadOnly' be modified?
+1
Search
Add
Resource group
Subscription
Refresh
Feedback
Diagnose and solve problems
Resource visualizer
Investigate
Monitoring
Alerts
Metrics
Diagnostic settings
Logs
Workbooks
Dashboards with Grafana
Usage
Configure
Settings
Locks
Automation
Help
Lock name
Lock type
Scope
Notes
protect-from-delete
Delete
func-finops-orchestrator-shamn
Edit
Delete
https://portal.azure.com/#@Ramshinaraoutlook.onmicrosoft.com/resource/subscriptions/cabcdcb1-5b0b-4b56-9e66-f487c8ffd899/resourceGroups/rg-finops-compliance-prod-Shamnan/providers/microsoft.insights/components/func-fi...

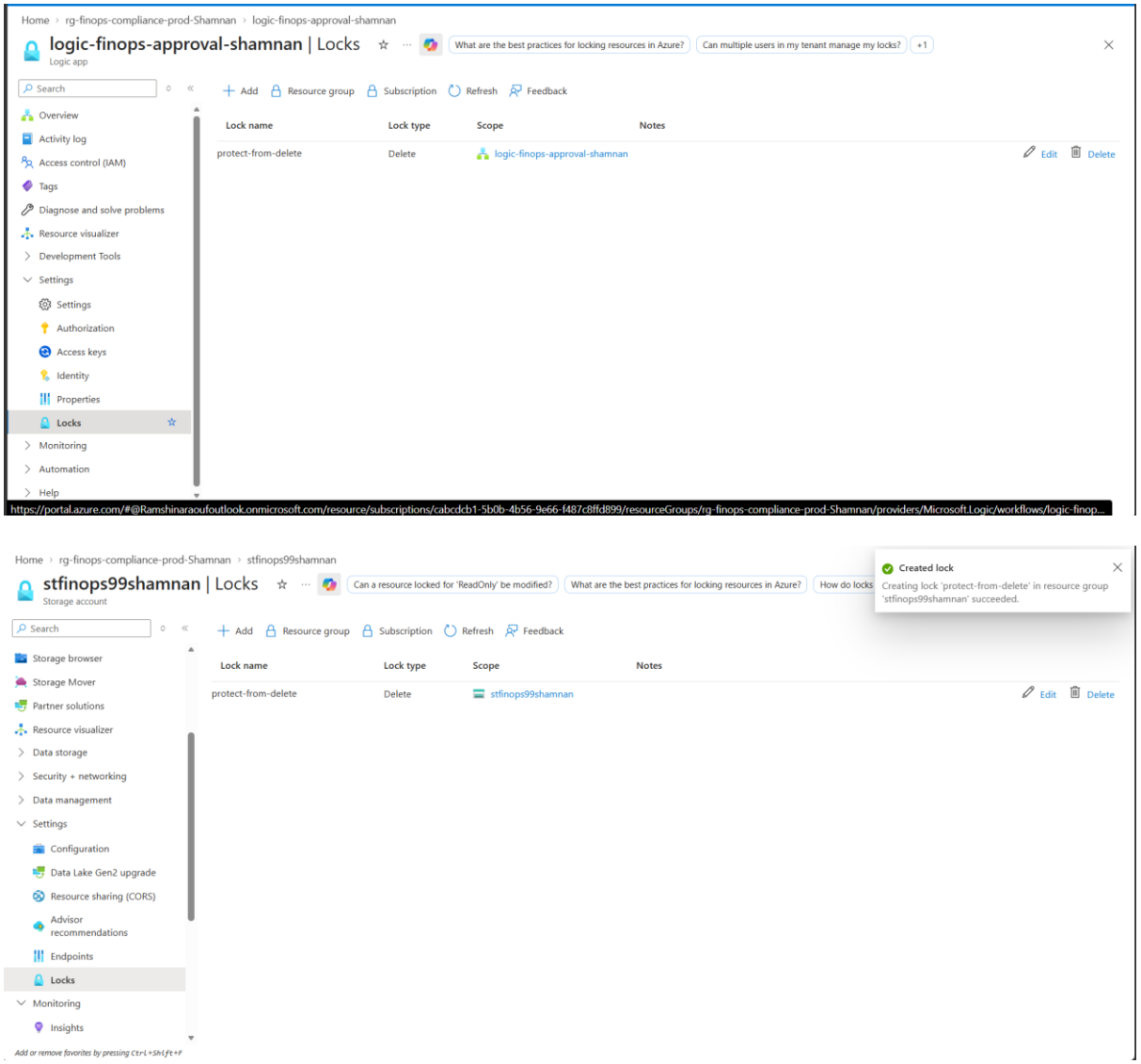


Figure 45: Locks page of one protected resource (protect-from-delete, Delete)

6. Testing and Validation

Each test below can be repeated for a live demonstration. Fill in the result and the figure number after you run it. Sample events are in Appendix B.

ID	Scenario	Action	Expected result	Result / Figure
T1	Untagged resource	Create a storage account with no tags in the resource group	Department = IT appears within about a minute; AutoRemediated record	[ ]
T2	Already tagged resource	Send the event for a tagged resource	No change, no new tag	[ ]
T3	Virtual machine	Send the VM event	Approval email arrives; ApprovalRequested record	[ ]
T4	Approve	Click Approve in the email	Remediation APPROVED email; Logic App Condition = true	[ ]
T5	Reject	Click Reject in a new email	Remediation REJECTED email; False branch green	[ ]
T6	Dry run	Mode dryrun, create public IP	WouldDelete record; IP remains	[ ]
T7	Keep tag	Create public IP with Keep = true	DeleteSkipped record; IP remains	[ ]
T8	Automatic delete	Mode on, create public IP	IP deleted; AutoDeleted record	[ ]
T9	Outside scope	Create a resource in another resource group	Ignored by the system	[ ]
T10	Failure alert	Send event for a non-existent storage account	Alert email arrives	[ ]
T11	Audit trail	Open Cosmos DB Items	Records for T1 to T8 present	[ ]
T12	Key Vault reference	Check LOGICAPP_URL setting	Reference resolved; approval email still sent	[ ]
T13	Dashboard	Open the web app	Records and totals displayed	[ ]

6.1 Evidence for the main scenarios

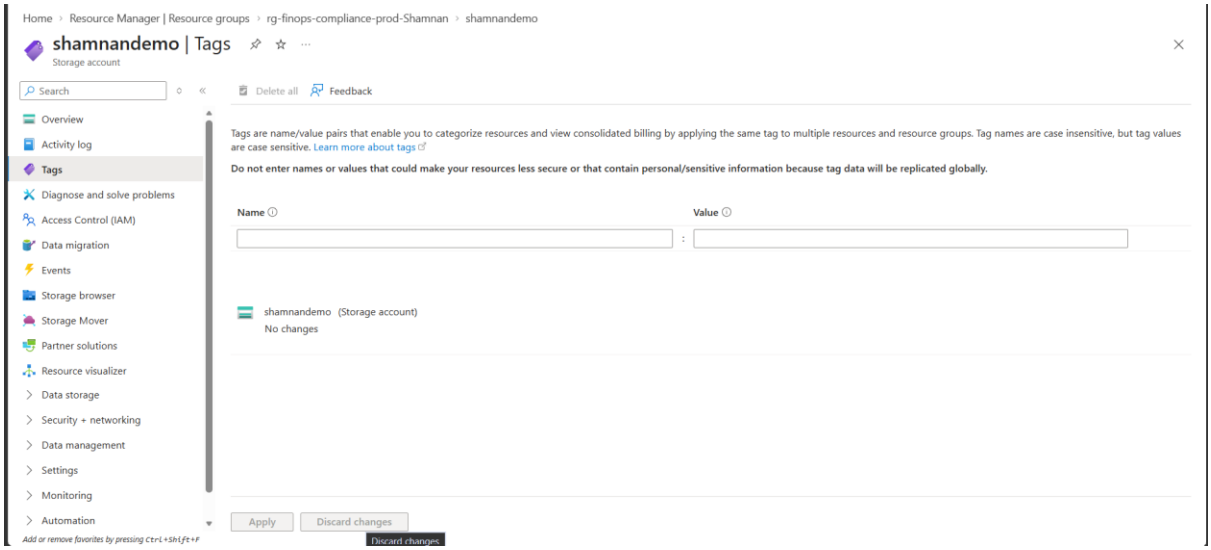


Figure 46: T1: test storage account with no tags before remediation (Tags page: No changes)

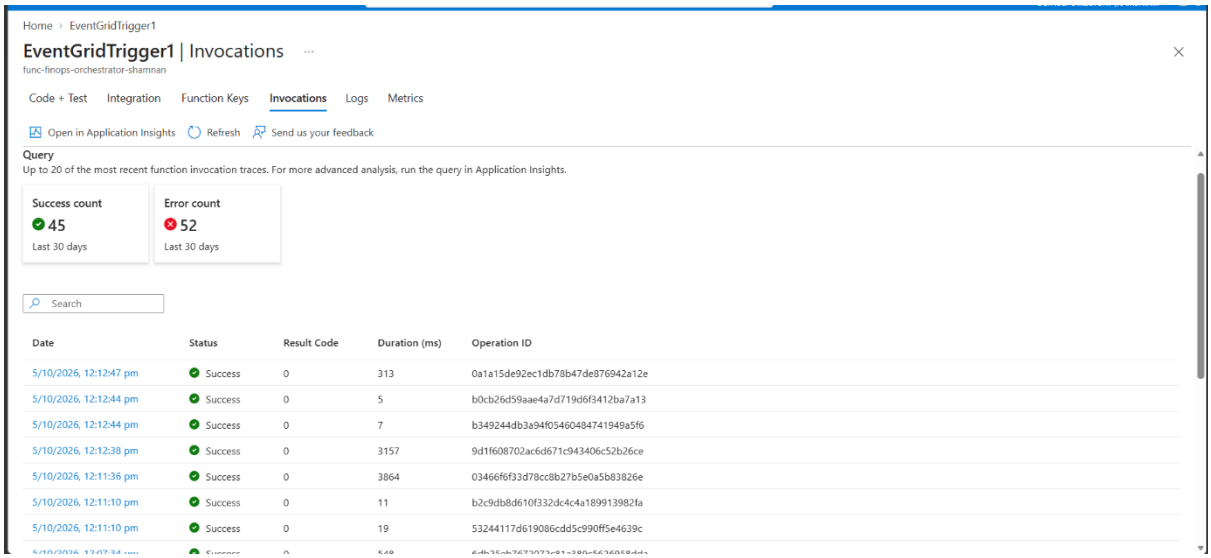


Figure 47: T1: Function Invocations and logs showing the event and 'Successfully remediated'

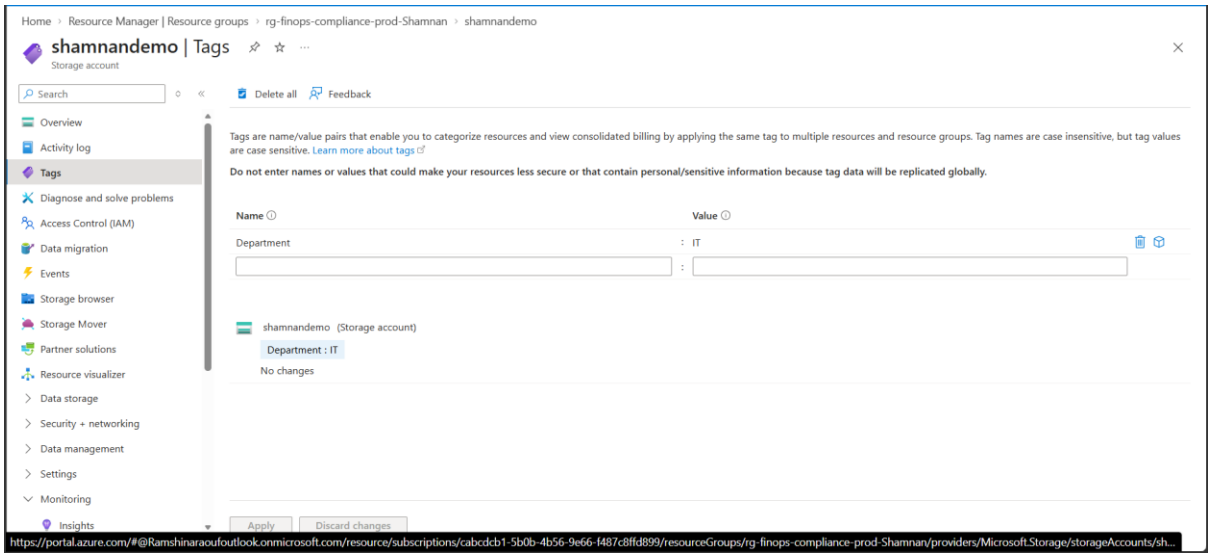


Figure 48: T1: Tags page after remediation: Department : IT

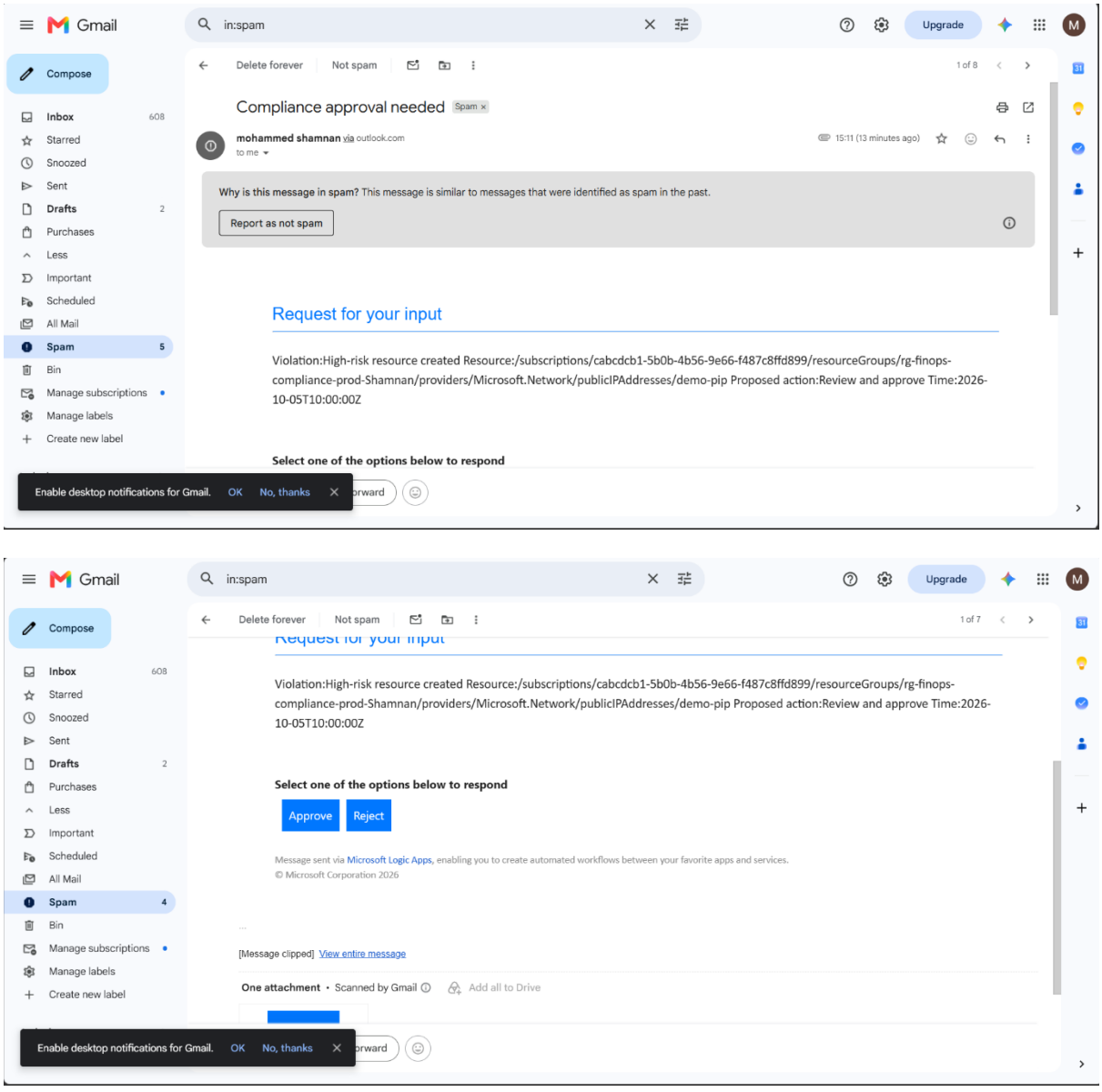


Figure 49: T3: approval email received with Approve and Reject buttons

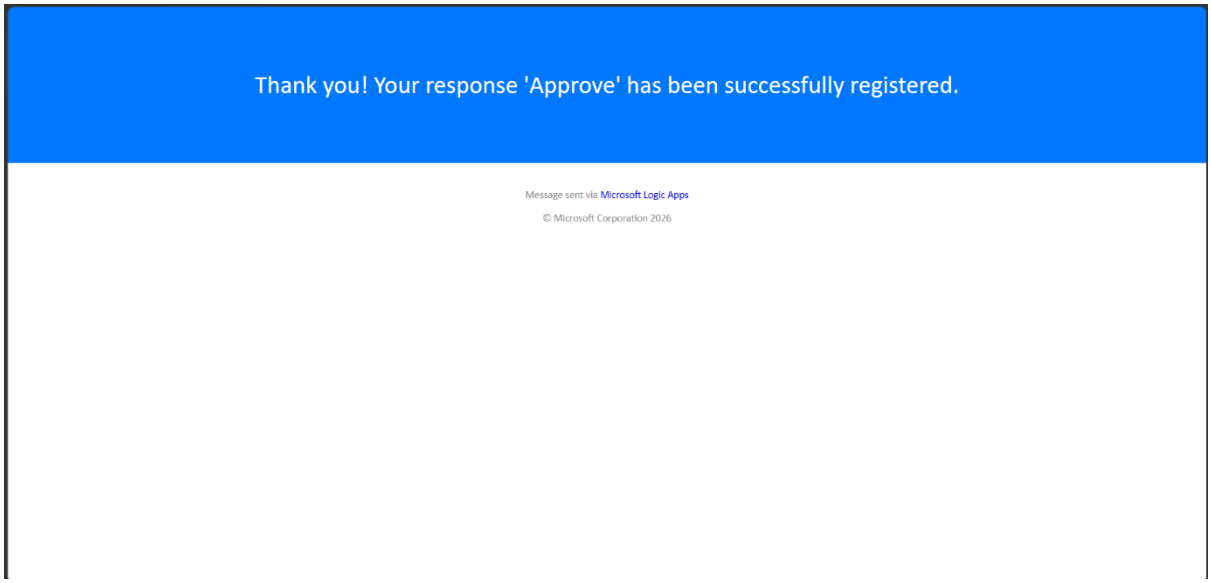


Figure 50: T4: Logic Apps confirmation page 'Your response Approve has been successfully registered'

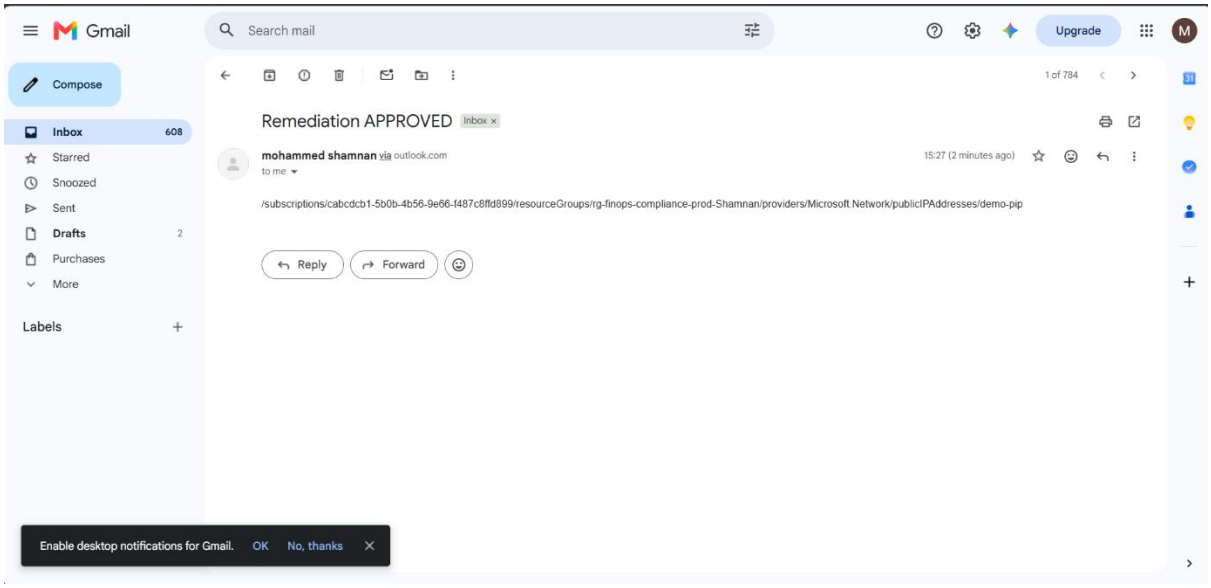
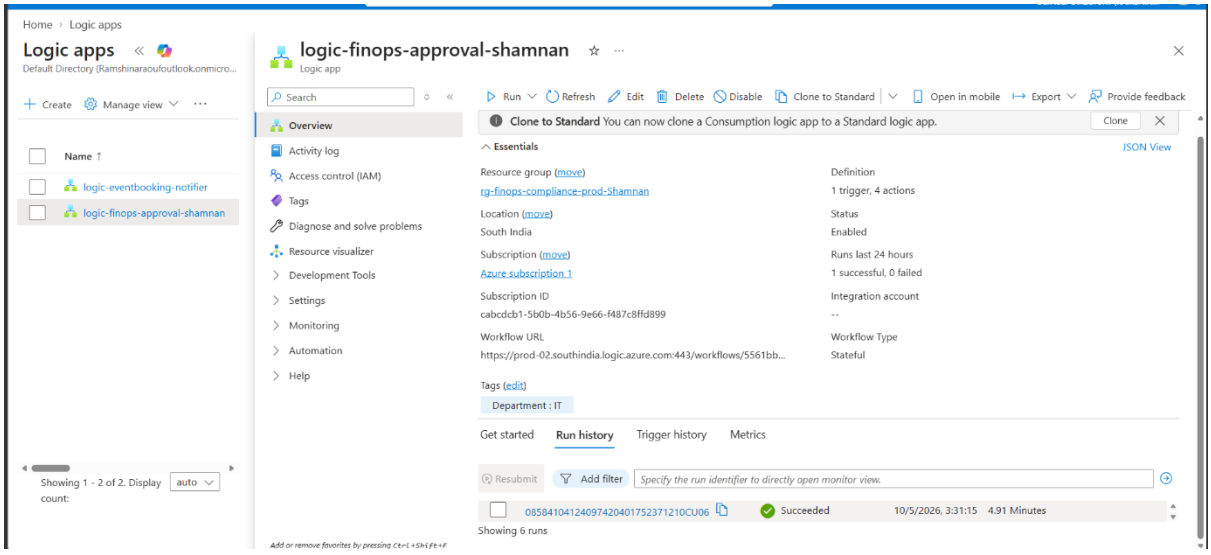
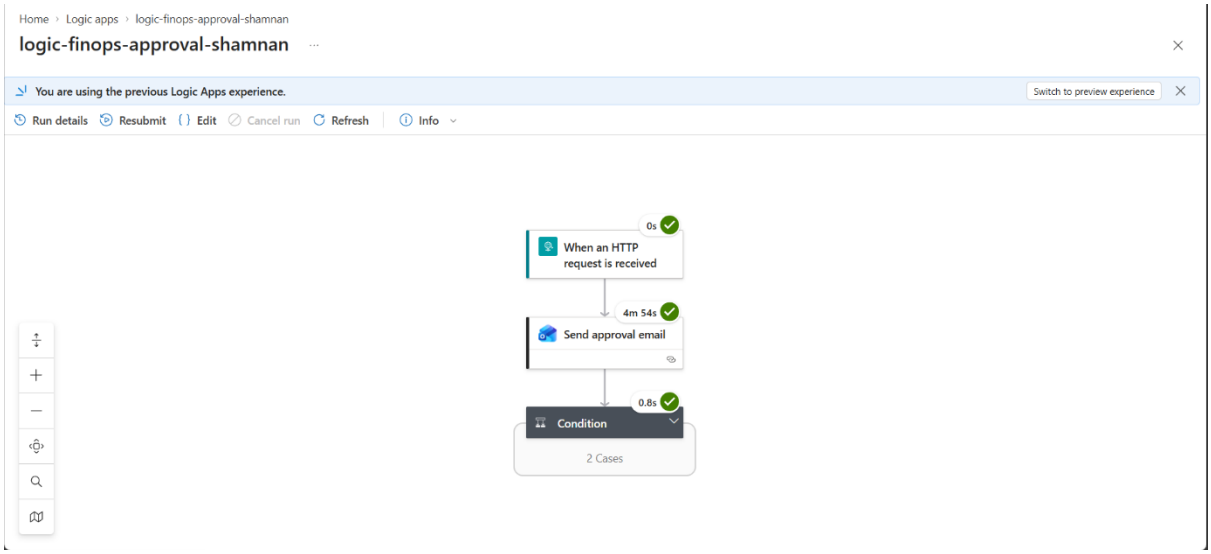


Figure 51: T4: Remediation APPROVED email received





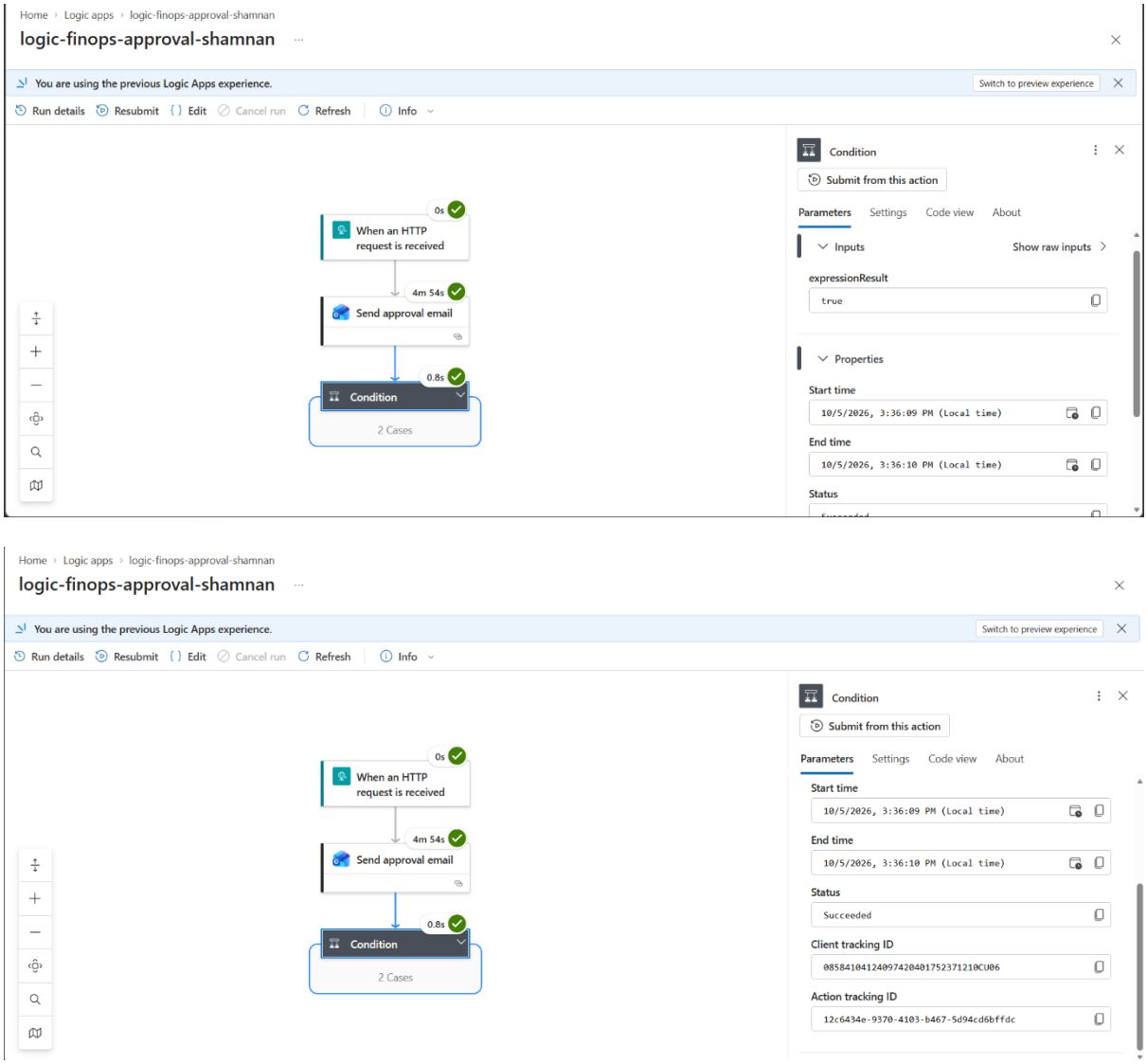
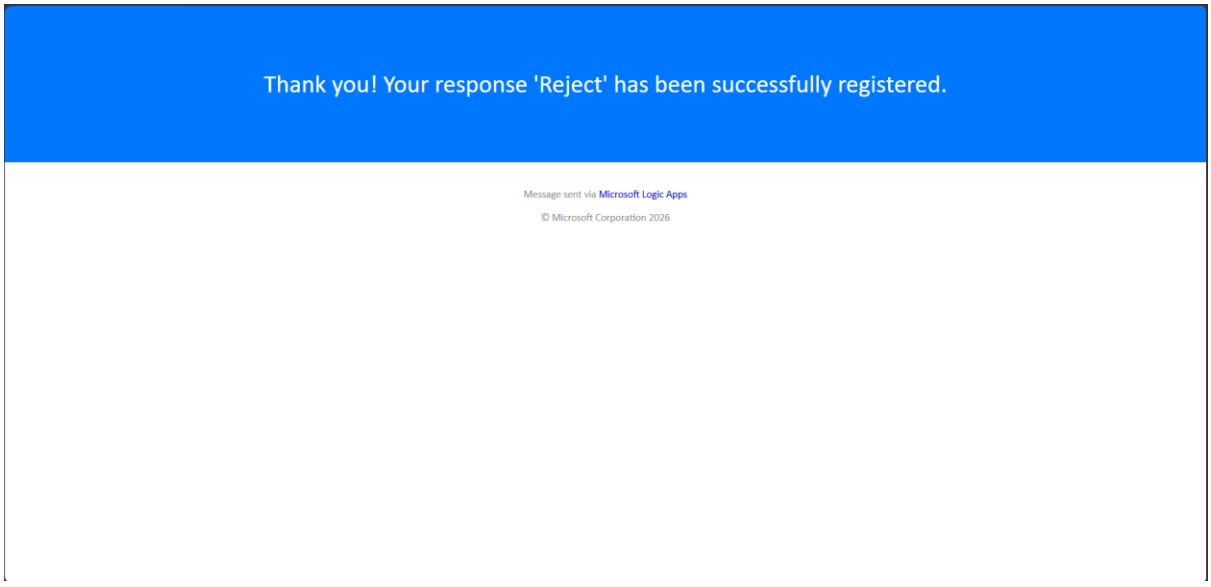
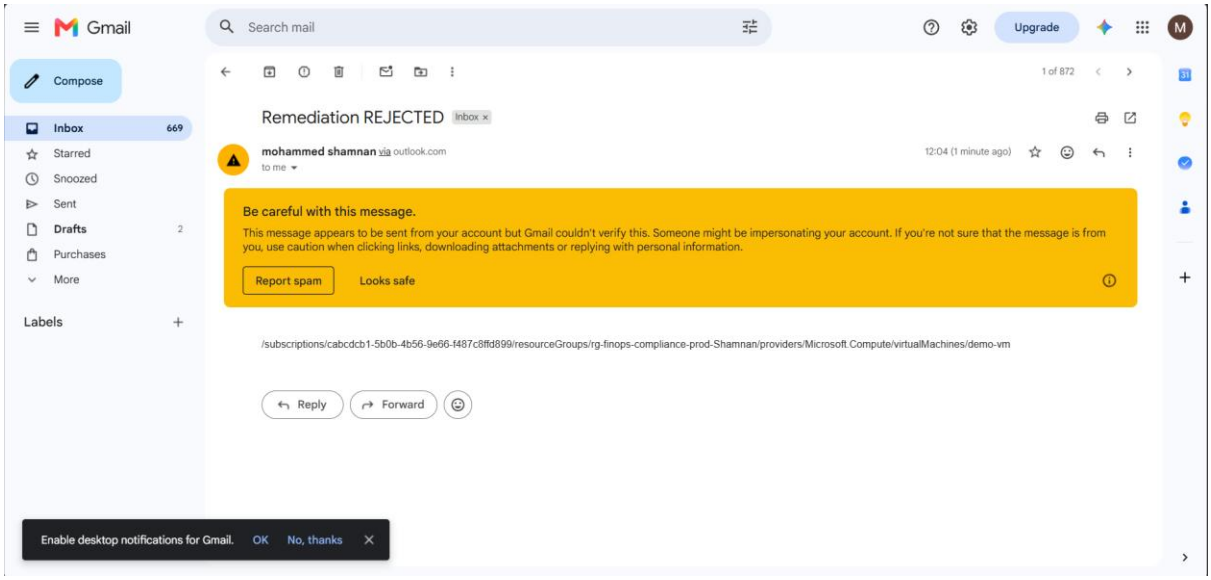


Figure 52: T4: Logic App run history, all steps green and Condition = true





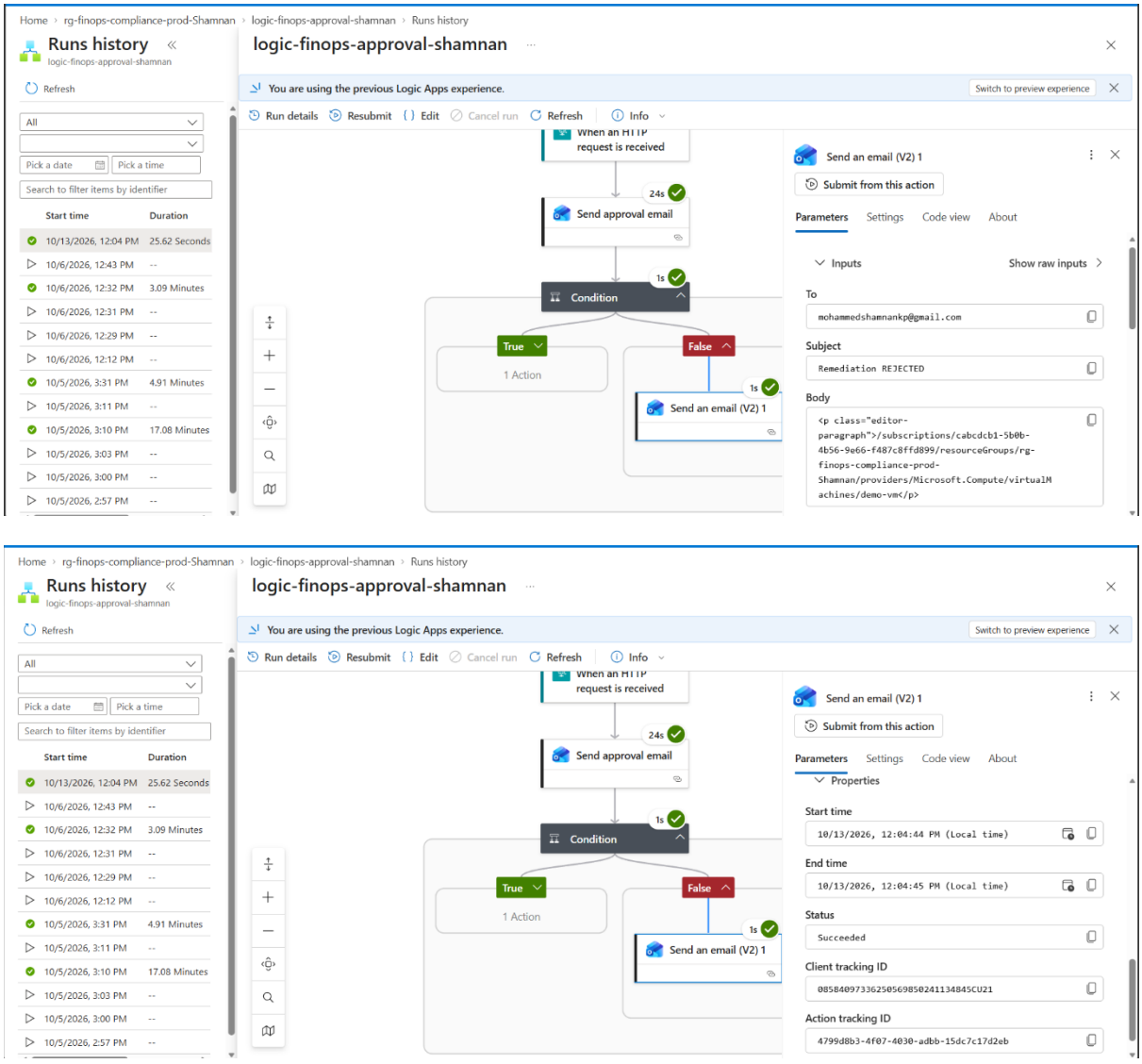


Figure 53: T5: Remediation REJECTED email and False branch in run history

7. Troubleshooting Log

Problems met during the build, their causes and the fixes. This log shows how each issue was diagnosed.

#	Symptom	Cause	Fix
1	Tags page shows 'Failed to fetch tags from ARG' and no results	Portal display error, not a missing tag	Check the Overview page or run Get-AzTag in Cloud Shell.
2	Test/Run returns 202 Accepted but the tag is not applied	Function failed silently. Application Insights was off, so errors were hidden	Turn on Application Insights and read the log or Invocations page.
3	Log shows 'The term Get-AzTag is not recognized'	The Az module was not loaded. requirements.psd1 had the Az line commented out	Enable Az.Accounts and Az.Resources, save, restart, wait for the first run to download modules.
4	Test/Run succeeds in under a second and prints nothing	The Body box was empty, so the function returned immediately	Paste a test event into the Body before clicking Run.
5	Approval email is in Spam	New automated sender	Click 'Not spam' once.
6	Role removal failed: scope is locked	A lock existed on the resource group	Remove the resource-group lock and place locks on individual resources instead.
7	Cannot create secret in Key Vault	Vault uses RBAC and the administrator had no data-plane role	Assign Key Vault Secrets Officer to the administrator.
8	Logic App URL appeared outside Azure	A URL with its secret key was shared	Regenerate the Logic App access key and update the Key Vault secret.
9	A mistyped AUTO_DELETE_MODE could have enabled deletion	Original condition checked only for not equal to off	Condition now requires the exact values dryrun or on.
10	Fake public IP event no longer sends an approval email	In dryrun or on mode public IPs follow the delete path	Use the virtual machine event for the approval demonstration.

8. Security Design

- **No stored credentials.** The function and the web app use managed identities. The only secret (the Logic App URL) is in Key Vault and referenced from settings.
- **Least privilege.** The function holds Tag Contributor, a custom public-IP-only delete role, a Cosmos DB data role and Key Vault Secrets User. The broad Contributor role was removed.
- **Scope guard.** The code ignores everything outside the project resource group.
- **Human approval** for virtual machines and, when automatic delete is off, for public IPs.
- **Protection against accidents:** delete locks, Keep tag, dry-run mode and a kill switch.
- **Network exposure:** the dashboard is restricted to the administrator IP address.
- **Full audit trail** in Cosmos DB and failure alerts in Azure Monitor.

9. Cost and Clean-up

9.1 Cost behaviour

Service	Billing model	Idle cost
Event Grid	Per operation	None
Azure Functions (Consumption)	Per execution	None
Cosmos DB (serverless)	Per request and storage used	Very low
Logic Apps (Consumption)	Per action run	None
Key Vault	Per operation	Very low
Application Insights	Per data ingested	Very low
App Service (Free F1)	Free tier	None
Container Instances	Per second while running	None after deletion
Defender Foundational CSPM	Free	None
Log alert rule	Small monthly fee per rule	Small. Delete after the demo if not needed
Standard public IP	Per hour while it exists	Delete test IPs the same day

9.2 Clean-up after the project

1. Set `AUTO_DELETE_MODE` to `off`.
2. Delete any test public IPs and the container group.
3. Regenerate the Logic App access key and update the Key Vault secret if the URL was ever shared.
4. Remove all resource locks first, otherwise the resource group cannot be deleted.
5. Delete the resource group to remove every resource at once.

10. Limitations and Future Work

10.1 Current limitations

- Approving a request currently sends a confirmation email and records the decision; it does not yet execute a remediation action.
- Automatic deletion is limited to public IP addresses. Virtual machines always need approval.
- The Function App uses the classic Consumption plan on Windows, which does not support virtual network integration. The network isolation described in the original proposal (private endpoints) is therefore not implemented.
- Resources are spread over several regions (South India, Central US, West US 2) because of regional availability on the subscription.
- One resource creation can emit several write events, so more than one audit record may appear for it.
- Approvals use email through the Outlook.com connector rather than Microsoft Teams.

10.2 Future work

- Execute a real remediation (for example stop or delete) when an administrator clicks Approve.
- Add more policies: public blob access on storage accounts, unencrypted disks, mandatory Environment tag.
- Use an ExpiresOn tag with a daily timer function that asks for approval before deleting expired resources.
- Package the deployment as Bicep templates with a CI/CD pipeline for repeatable environments.
- Move to Flex Consumption with VNet integration and private endpoints for Cosmos DB and Key Vault.
- Send approvals to Microsoft Teams and add monthly cost reports.

11. Conclusion

The project delivers a working governance engine built entirely from serverless Azure services. It detects drift within seconds, fixes low-risk problems automatically, asks a human before touching anything risky, can remove unapproved public IP addresses under strict safeguards, and keeps a complete audit trail with monitoring and a dashboard. The design follows least-privilege and safe-by-default principles, and every step in this document can be reproduced from the instructions and verified from the evidence screenshots.

Appendix A: Final run.ps1 (EventGridTrigger1)

Paste this script into **Functions > EventGridTrigger1 > Code + Test**.

```
param($eventGridEvent, $TriggerMetadata)

function Write-AuditLog {
    param($ResourceId, $Violation, $Action, $Status)
    try {
        $account = $env:COSMOS_ENDPOINT.TrimEnd('/')
        $uri = "$account/dbs/$($env:COSMOS_DB)/colls/$($env:COSMOS_CONTAINER)/docs"

        $token = (Get-AzAccessToken -ResourceUrl $account).Token
        if ($token -is [System.Security.SecureString]) {
            $token = [System.Net.NetworkCredential]::new('', $token).Password
        }

        $rg = if ($ResourceId -match "/resourceGroups/([^/]+)/") { $Matches[1] } else {
"unknown" }

        $doc = @{
            id            = [guid]::NewGuid().ToString()
            resourceId    = $ResourceId
            resourceGroup = $rg
            violation     = $Violation
            action        = $Action
            status        = $Status
            timestamp     = (Get-Date).ToUniversalTime().ToString("o")
        } | ConvertTo-Json

        $headers = @{
            "Authorization"               =
[uri]::EscapeDataString("type=aad&ver=1.0&sig=$token")
            "x-ms-date"                   = [DateTime]::UtcNow.ToString("r")
            "x-ms-version"                = "2018-12-31"
            "x-ms-documentdb-partitionkey" = "[""$ResourceId""]"
        }

        Invoke-RestMethod -Method Post -Uri $uri -Headers $headers -Body $doc -ContentType
"application/json" -SkipHeaderValidation -ErrorAction Stop | Out-Null
        "Audit record saved to Cosmos DB." | Out-Host
    }
    catch {
        Write-Warning "Audit log failed (main flow continues): $_"
    }
}

"Incoming Event Grid Payload:" | Out-Host
$eventGridEvent | Out-String | Out-Host

if ($eventGridEvent -is [string]) { $eventObject = $eventGridEvent | ConvertFrom-Json } else {
$eventObject = $eventGridEvent }

$eventType = $eventObject.eventType
$data = $eventObject.data
if ($eventType -ne "Microsoft.Resources.ResourceWriteSuccess") { return }

$resourceId = $data.resourceUri
if (-not $resourceId) { $resourceId = $eventObject.subject }

if ($resourceId -notmatch "/resourceGroups/rg-finops-compliance-prod-Shamnan/providers/") {
    "Outside target scope, skipping: $resourceId" | Out-Host
    return
}
if ($resourceId -match "/providers/Microsoft.Resources/deployments/") {
```

```
    "Deployment event, skipping." | Out-Host
    return
}

# AUTOMATIC DELETE PATH (public IPs only; controlled by AUTO_DELETE_MODE: off | dryrun | on)
$mode = if ($env:AUTO_DELETE_MODE) { $env:AUTO_DELETE_MODE.ToLower() } else { "off" }

if ($resourceId -match "/providers/Microsoft.Network/publicIPAddresses/[^/]+$" -and ($mode -eq
"dryrun" -or $mode -eq "on")) {
    try {
        $tags = $null
        try {
            $tags = (Get-AzTag -ResourceId $resourceId -ErrorAction
Stop).Properties.TagsProperty
        }
        catch {
            if ("$_" -match "NotFound|could not be found|ResourceNotFound") {
                "Resource no longer exists, nothing to do." | Out-Host
                return
            }
            throw
        }

        if ($tags -and $tags.ContainsKey("Keep") -and $tags["Keep"] -eq "true") {
            "Keep=true tag present. Not deleting." | Out-Host
            Write-AuditLog -ResourceId $resourceId -Violation "Unapproved public IP" -Action
"DeleteSkipped" -Status "KeepTag"
            return
        }

        if ($mode -eq "dryrun") {
            "DRY RUN: would delete $resourceId" | Out-Host
            Write-AuditLog -ResourceId $resourceId -Violation "Unapproved public IP" -Action
"WouldDelete" -Status "DryRun"
            return
        }

        "Deleting public IP: $resourceId" | Out-Host
        $resp = Invoke-AzRestMethod -Method DELETE -Path "$($resourceId)?api-version=2023-09-
01"

        if ($resp.StatusCode -eq 404) {
            "Already deleted, nothing to do." | Out-Host
            return
        }
        if ($resp.StatusCode -ge 200 -and $resp.StatusCode -lt 300) {
            "Deleted: $resourceId" | Out-Host
            Write-AuditLog -ResourceId $resourceId -Violation "Unapproved public IP" -Action
"AutoDeleted" -Status "Success"
        }
        else {
            throw "Delete returned status $($resp.StatusCode): $($resp.Content)"
        }
    }
    catch {
        Write-AuditLog -ResourceId $resourceId -Violation "Unapproved public IP" -Action
"AutoDeleted" -Status "Failed"
        Write-Error "Auto-delete failed: $_"
        throw
    }
    return
}

# HIGH-RISK PATH (VMs always; public IPs too when AUTO_DELETE_MODE is off)
if ($resourceId -match
"/providers/Microsoft.(Network/publicIPAddresses|Compute/virtualMachines)/") {
    "High-risk resource detected. Requesting approval..." | Out-Host
    $body = @{
```

```
        resourceId     = $resourceId
        violation      = "High-risk resource created"
        proposedAction = "Review and approve"
        eventTime      = $eventObject.eventTime
    } | ConvertTo-Json

    try {
        Invoke-RestMethod -Method Post -Uri $env:LOGICAPP_URL -Body $body -ContentType
"application/json" -ErrorAction Stop | Out-Null
        "Approval request sent to Logic App." | Out-Host
        Write-AuditLog -ResourceId $resourceId -Violation "High-risk resource created" -Action
"ApprovalRequested" -Status "Pending"
    }
    catch {
        Write-AuditLog -ResourceId $resourceId -Violation "High-risk resource created" -Action
"ApprovalRequested" -Status "Failed"
        Write-Error "Failed to call Logic App: $_"
        throw
    }
    return
}

"Processing resource: $resourceId" | Out-Host

# LOW-RISK PATH
try {
    $existingTags = (Get-AzTag -ResourceId $resourceId -ErrorAction
Stop).Properties.TagsProperty
    if ($existingTags -and $existingTags.ContainsKey("Department")) {
        "Resource already complies with Department tag standards." | Out-Host
        return
    }
    "Department tag missing. Applying auto-remediation..." | Out-Host
    Update-AzTag -ResourceId $resourceId -Tag @{ Department = "IT" } -Operation Merge -
ErrorAction Stop
    "Successfully remediated resource: $resourceId with Department: IT" | Out-Host
    Write-AuditLog -ResourceId $resourceId -Violation "MissingDepartmentTag" -Action
"AutoRemediated" -Status "Success"
}
catch {
    Write-AuditLog -ResourceId $resourceId -Violation "MissingDepartmentTag" -Action
"AutoRemediated" -Status "Failed"
    Write-Error "Failed to process resource remediation: $_"
    throw
}
```

Appendix B: Sample Test Events

Paste one of these into **Test/Run > Input > Body** and click **Run**. Replace `<subscription-id>` with your subscription ID. Check that the Body is not empty before running.

B.1 Storage account without a Department tag (auto-tagging test)

```
{
  "id": "3",
  "eventType": "Microsoft.Resources.ResourceWriteSuccess",
  "subject": "/subscriptions/<subscription-id>/resourceGroups/rg-finops-compliance-prod-
Shamnan/providers/Microsoft.Storage/storageAccounts/shamnandemo",
  "data": {
    "resourceUri": "/subscriptions/<subscription-id>/resourceGroups/rg-finops-compliance-prod-
Shamnan/providers/Microsoft.Storage/storageAccounts/shamnandemo"
  },
  "eventTime": "2026-10-05T10:00:00Z",
  "dataVersion": "1",
  "metadataVersion": "1",
  "topic": "/subscriptions/<subscription-id>"
}
```

B.2 Virtual machine (approval email test)

```
{
  "id": "5",
  "eventType": "Microsoft.Resources.ResourceWriteSuccess",
  "subject": "/subscriptions/<subscription-id>/resourceGroups/rg-finops-compliance-prod-
Shamnan/providers/Microsoft.Compute/virtualMachines/demo-vm",
  "data": {
    "resourceUri": "/subscriptions/<subscription-id>/resourceGroups/rg-finops-compliance-prod-
Shamnan/providers/Microsoft.Compute/virtualMachines/demo-vm"
  },
  "eventTime": "2026-10-05T10:00:00Z",
  "dataVersion": "1",
  "metadataVersion": "1",
  "topic": "/subscriptions/<subscription-id>"
}
```

B.3 Public IP (automatic-delete path)

```
{
  "id": "2",
  "eventType": "Microsoft.Resources.ResourceWriteSuccess",
  "subject": "/subscriptions/<subscription-id>/resourceGroups/rg-finops-compliance-prod-
Shamnan/providers/Microsoft.Network/publicIPAddresses/demo-pip",
  "data": {
    "resourceUri": "/subscriptions/<subscription-id>/resourceGroups/rg-finops-compliance-prod-
Shamnan/providers/Microsoft.Network/publicIPAddresses/demo-pip"
  },
  "eventTime": "2026-10-05T10:00:00Z",
  "dataVersion": "1",
  "metadataVersion": "1",
  "topic": "/subscriptions/<subscription-id>"
}
```

B.4 Non-existent storage account (deliberate failure, alert test)

```
{
  "id": "4",
  "eventType": "Microsoft.Resources.ResourceWriteSuccess",
  "subject": "/subscriptions/<subscription-id>/resourceGroups/rg-finops-compliance-prod-
Shamnan/providers/Microsoft.Storage/storageAccounts/doesnotexist123",
  "data": {
    "resourceUri": "/subscriptions/<subscription-id>/resourceGroups/rg-finops-compliance-prod-
Shamnan/providers/Microsoft.Storage/storageAccounts/doesnotexist123"
  },
  "eventTime": "2026-10-05T10:00:00Z",
  "dataVersion": "1",
```

```
  "metadataVersion": "1",
  "topic": "/subscriptions/<subscription-id>"
}
```

Appendix C: Dashboard Reference Code (app.py)

Reference implementation of the dashboard. `requirements.txt` contains: flask, gunicorn, azure-identity, azure-cosmos.

```
import os
from collections import Counter
from flask import Flask, render_template_string
from azure.identity import DefaultAzureCredential
from azure.cosmos import CosmosClient

app = Flask(__name__)
client = CosmosClient(os.environ["COSMOS_ENDPOINT"], credential=DefaultAzureCredential())
container =
client.get_database_client(os.environ["COSMOS_DB"]).get_container_client(os.environ["COSMOS_CO
NTAINER"])

PAGE = """
<!doctype html><title>FinOps Compliance Dashboard</title>
<style>
 body{font-family:Segoe UI,Arial;margin:30px;background:#f4f6f8}
 .cards{display:flex;gap:16px;margin-bottom:24px}
 .card{background:#fff;padding:16px 24px;border-radius:8px;box-shadow:0 1px 3px #0002}
 .card b{font-size:28px;display:block}
 table{border-collapse:collapse;width:100%;background:#fff}
 th,td{padding:8px 12px;border-bottom:1px solid #ddd;text-align:left;font-size:14px}
 th{background:#0078d4;color:#fff}
</style>
<h1>FinOps Compliance Dashboard</h1>
<div class="cards">
 <div class="card"><b>{{ total }}</b>Total records</div>
 {% for k,v in actions.items() %}<div class="card"><b>{{ v }}</b>{{ k }}</div>{% endfor %}
</div>
<table><tr><th>Time
(UTC)</th><th>Resource</th><th>Violation</th><th>Action</th><th>Status</th></tr>
{% for r in rows %}<tr><td>{{ r.timestamp[:19] }}</td><td>{{ r.resourceId.split('/')[-1]
}}</td><td>{{ r.violation }}</td><td>{{ r.action }}</td><td>{{ r.status }}</td></tr>{% endfor
%}
</table>
"""

@app.route("/")
def home():
    rows = list(container.query_items(
        query="SELECT TOP 100 * FROM c ORDER BY c.timestamp DESC",
        enable_cross_partition_query=True))
    return render_template_string(PAGE, rows=rows, total=len(rows),
                                  actions=Counter(r["action"] for r in rows))
```

Appendix D: Screenshot Evidence Checklist

Tick each box when the screenshot has been pasted into the matching dashed box.

Fig.	Screenshot	Section	Done
1	Architecture diagram of the solution (draw in PowerPoint or draw.io and paste here)	2.1 End-to-end flow	[ ]
2	Empty resource group before any resource was deployed	Phase 1: Resource group and environment	[ ]
3	Cosmos DB review page showing Serverless capacity mode and account name	Phase 2: Azure Cosmos DB audit store	[ ]
4	Data Explorer showing ComplianceDB > AuditLogs	Phase 2: Azure Cosmos DB audit store	[ ]
5	Key Vault review page (name, region, Standard tier)	Phase 3: Azure Key Vault	[ ]
6	Key Vault deployment succeeded	Phase 3: Azure Key Vault	[ ]
7	Assign policy review page (Require-Department-Tag)	Phase 4: Azure Policy guardrail	[ ]
8	Policy assignment details with parameters tagName and tagValue	Phase 4: Azure Policy guardrail	[ ]
9	Function App review page (PowerShell 7.6, Consumption, Windows)	Phase 5: Function App (remediation engine)	[ ]
10	Function App overview with Department: IT tag	Phase 5: Function App (remediation engine)	[ ]
11	requirements.psd1 with Az.Accounts and Az.Resources enabled	Phase 5: Function App (remediation engine)	[ ]
12	Identity page: system assigned managed identity switched On	Phase 5: Function App (remediation engine)	[ ]
13	Event Grid system topic review page (source = resource group)	Phase 6: Event Grid system topic and subscription	[ ]
14	Create Event Subscription form with Azure Function endpoint EventGridTrigger1	Phase 6: Event Grid system topic and subscription	[ ]
15	Topic overview: subscription sub-finops-remediation Succeeded, with event metrics	Phase 6: Event Grid system topic and subscription	[ ]
16	Role assignments list showing only Tag Contributor and Public IP Deleter (FinOps) for the function	Phase 7: Identities and permissions (RBAC)	[ ]
17	Hosting plan choice: Consumption, Multi-tenant	Phase 8: Logic App approval workflow	[ ]
18	Logic App designer: trigger, approval email, condition, True and False branches	Phase 8: Logic App approval workflow	[ ]
19	Send approval email settings (To, Subject, User Options, Body)	Phase 8: Logic App approval workflow	[ ]

Fig.	Screenshot	Section	Done
20	Condition: SelectedOption equals Approve	Phase 8: Logic App approval workflow	[ ]
21	Key Vault secrets list showing LogicAppApprovalUrl (value hidden)	Phase 9: Store the Logic App URL in Key Vault	[ ]
22	App setting LOGICAPP_URL showing a resolved Key Vault reference	Phase 9: Store the Logic App URL in Key Vault	[ ]
23	Cosmos DB Items list with audit records	Phase 10: Audit logging to Cosmos DB	[ ]
24	One expanded audit record (resourceId, violation, action, status, timestamp)	Phase 10: Audit logging to Cosmos DB	[ ]
25	Code + Test showing run.ps1 with the Write-AuditLog function and decision logic	Phase 11: Function logic in Code + Test	[ ]
26	Cloud Shell output after creating the Public IP Deleter (FinOps) role	Phase 12: Automatic delete option	[ ]
27	App setting AUTO_DELETE_MODE = dryrun	Phase 12: Automatic delete option	[ ]
28	run.ps1 showing the AUTOMATIC DELETE PATH block	Phase 12: Automatic delete option	[ ]
29	Test A: public IP created (review page) and still present after the dry run	Phase 12: Automatic delete option	[ ]
30	Test A: Cosmos DB record action WouldDelete, status DryRun	Phase 12: Automatic delete option	[ ]
31	Test B: public IP with Keep = true tag (review page)	Phase 12: Automatic delete option	[ ]
32	Test B: Cosmos DB record action DeleteSkipped, status KeepTag	Phase 12: Automatic delete option	[ ]
33	App setting AUTO_DELETE_MODE = on	Phase 12: Automatic delete option	[ ]
34	Test C: Cosmos DB record action AutoDeleted, status Success	Phase 12: Automatic delete option	[ ]
35	Resource group list after the tests (deleted IP is gone; protected IP remains)	Phase 12: Automatic delete option	[ ]
36	Alert rule configuration (query and threshold)	Phase 13: Monitoring and alerts (Azure Monitor)	[ ]
37	Action group with the email notification	Phase 13: Monitoring and alerts (Azure Monitor)	[ ]
38	Alert email received after the deliberate failure	Phase 13: Monitoring and alerts (Azure Monitor)	[ ]
39	Defender plans page with Foundational CSPM On and paid plans Off	Phase 14: Microsoft Defender for Cloud	[ ]
40	Secure score and one recommendation	Phase 14: Microsoft Defender for Cloud	[ ]

Fig.	Screenshot	Section	Done
41	Container group overview	Phase 15: Azure Container Instances vulnerability scan	[ ]
42	Container logs showing the vulnerability scan result	Phase 15: Azure Container Instances vulnerability scan	[ ]
43	Dashboard in the browser listing audit records and totals	Phase 16: Admin dashboard (Azure App Service)	[ ]
44	Web app networking restriction (access limited to the administrator IP)	Phase 16: Admin dashboard (Azure App Service)	[ ]
45	Locks page of one protected resource (protect-from-delete, Delete)	Phase 17: Resource locks	[ ]
46	T1: test storage account with no tags before remediation (Tags page: No changes)	6.1 Evidence for the main scenarios	[ ]
47	T1: Function Invocations and logs showing the event and 'Successfully remediated'	6.1 Evidence for the main scenarios	[ ]
48	T1: Tags page after remediation: Department : IT	6.1 Evidence for the main scenarios	[ ]
49	T3: approval email received with Approve and Reject buttons	6.1 Evidence for the main scenarios	[ ]
50	T4: Logic Apps confirmation page 'Your response Approve has been successfully registered'	6.1 Evidence for the main scenarios	[ ]
51	T4: Remediation APPROVED email received	6.1 Evidence for the main scenarios	[ ]
52	T4: Logic App run history, all steps green and Condition = true	6.1 Evidence for the main scenarios	[ ]
53	T5: Remediation REJECTED email and False branch in run history	6.1 Evidence for the main scenarios	[ ]